

[프로젝트3] MiniJava -to- MIPS 컴파일러

학과: 소프트웨어공학과

학번: 213652

이름: 김지환

1. 서론

1.1. 프로젝트 개요

본 프로젝트는 자바 언어의 서브셋인 MiniJava 소스코드를 입력받아, 어휘 및 구문 분석, 의미 분석 및 타입 검사, 중간 표현 트리 변환, 명령어 선택, 그리고 가점 요건인 활성 분석 및 레지스터 할당을 거쳐 최종적으로 32비트 MIPS 어셈블리 코드를 생성하는 전체 컴파일러 백엔드를 구현하는 것을 목적으로 한다.

1.2. MiniJava 언어 명세 및 MIPS 아키텍처 특징

MiniJava는 클래스 선언, 멤버 변수 상속, 이항 연산, 일차원 배열, 제어문(if, while) 및 정수 출력(System.out.println)을 지원하는 소형 객체지향 언어이다. 출력 문법상 정수 타입 한 개만을 매개 변수로 받아 출력할 수 있는 명세적 한계를 지닌다. 타겟 아키텍처인 MIPS는 32개의 범용 레지스터를 사용하는 RISC 구조이며, 본 컴파일러는 가상 레지스터를 MIPS의 물리 범용 레지스터(s0-s7, t0-t9 등)로 최적 매핑하는 할당 아키텍처를 목표로 한다.

1.3. 컴파일러 전체 아키텍처 요약

본 컴파일러는 아래와 같은 다단계 파이프라인으로 구성되어 데이터 스트림을 순차적으로 변환한다.

- 프론트엔드 (Parser & TypeCheck): 소스코드 검증 및 추상 구문 트리(AST) 형성
- 미들엔드 (Translate & Canon): AST를 프레임 레이아웃 오프셋이 적용된 IR Tree(Stm/Exp)로 변환 후, Side-Effect를 제거한 순차적 기본 블록 및 트레이스(Traces)로 정규화
- 백엔드 (Codegen & RegAlloc): Maximal Munch 타일 매칭 기반 MIPS 어셈블리 명령어 방출 및 간섭 그래프 채색 기반 레지스터 할당

1.4. 개발 환경 구성 및 도구

컴파일러 구현 및 빌드: JDK 21.0.10 (Java SE 환경)

대상 시뮬레이터: MARS (MIPS Assembler and Runtime Simulator) v4.5 jar

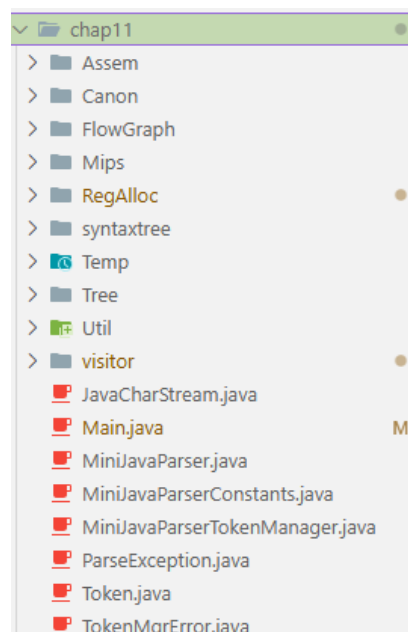
자동화 테스트: Windows PowerShell 환경에서의 자동 회귀 검증 스크립트 연동

2. 초기 패키지 구성 및 빌드 오류 해결 (Chap 6)

2.1. 컴파일러 소스 통합 및 패키지 구조 단일화

본 프로젝트는 5장에서 구현한 타입 검사기(Type Checker) 소스코드를 베이스로 시작한다. 여기에 교수님께서 제공해 주신 MIPS Frame 소스(Chap 6), Canonical Tree / Basic Blocks / Traces 소스(Chap 8)와 Appel 교재의 나머지 단계별 뼈대 소스코드들을 단 하나의 폴더 구조로 취합하기 위해, 루트 디렉토리에 chap11 폴더를 생성하고 모든 파일을 chap11 디렉토리로 모아 통합하였다.

통합 후 패키지 간의 참조 관계를 일치시키고 임포트 오류를 방지하기 위해, 각 소스코드 상단의 패키지 선언을 package chap11.Temp;, package chap11.Mips; 등 chap11을 루트로 하는 단일 패키지 체계로 정리하여 네임스페이스를 일원화하였다.



2.2. 프레임 레이아웃 검증 및 빌드 오류 해결

이송 및 패키지화가 올바르게 수행되었는지 1차 검증하기 위해, MIPS 스택 프레임 레이아웃 클래스들을 검증하는 Mips.FrameDriver 단독 컴파일 및 구동을 시도하였다. 그러나 교재 원본 뼈대 자체의 의존성 오류들로 인해 컴파일이 실패하였다.

```
symbol:   class Label
location: class Temp
chap11\Temp\Label.java:31: error: reference to Label is ambiguous
    this("L" + count++);
    ^
    both constructor Label(String) in Label and constructor Label(Symbol) in Label match
chap11\Tree\JUMP.java:9: error: cannot find symbol
    this(new NAME(target), new Temp.LabelList(target,null));
    ^
symbol:   class LabelList
location: class Temp
11 errors
```

이를 해결하기 위해 아래와 같이 디버깅 및 소스 수정을 진행하여 최초로 빌드가 성공적으로 통과되도록 안정화하였다.

에러 1: Symbol 패키지 누락 및 의존성 에러

- 원인: Label.java에서 존재하지 않는 Symbol 패키지를 참조하여 컴파일 실패.

```
minijava_compiler > chap11 > Temp >  Label.java
1 package Temp;
2 import Symbol.Symbol;

37
38 public Label(Symbol s) {
39     this(s.toString());
40 }
```

- 해결: Label.java 내부에서 불필요한 Symbol импорт 구문 및 관련 생성자를 삭제하여 의존성을 정제함.

```
1 package Temp;
2
```

에러 2: Temp 패키지명과 클래스명 충돌(Shadowing)

- 원인: TempMap.java 등에서 패키지 이름 Temp와 클래스 이름 Temp가 동일하여, 타입 명시 시 컴파일러가 클래스 식별에 실패함.

```
minijava_compiler > chap11 > Temp >  TempMap.java
1 package Temp;
2
3 public interface TempMap {public String tempMap(Temp.Temp t);}
4
5
```

- 해결: Temp.Temp 형태의 명칭을 Temp 단축형으로 변경하여 새도잉 문제를 제거함.

```
1 package Temp;
2
3 public interface TempMap {public String tempMap(Temp t);}
4
```

에러 3: JUMP.java 내 Temp 패키지 참조 꼬임

- 원인: JUMP.java 내에서 패키지명 새도잉으로 인해 Temp.LabelList를 식별하지 못해 빌드 실패.

```

1 package Tree;
2 import Temp.Temp;
3 import Temp.Label;
4 public class JUMP extends Stm {
5     public Exp exp;
6     public Temp.Labellist targets;
7     public JUMP(Exp e, Temp.Labellist t) {exp=e; targets=t;}
8     public JUMP(Temp.Label target) {
9         this(new NAME(target), new Temp.Labellist(target,null));
10    }
11    public ExpList kids() {return new Explist(exp,null);}
12    public Stm build(Explist kids) {
13        return new JUMP(kids.head,targets);
14    }
15 }

```

- 해결: Temp.Labellist 대신 Temp.LabelList를 명시적으로 개별 import하여 타입을 인식시킴.

```

1 package Tree;
2 import Temp.Temp;
3 import Temp.Label;
4 import Temp.LabelList;
5 public class JUMP extends Stm {
6     public Exp exp;
7     public LabelList targets;
8     public JUMP(Exp e, LabelList t) {exp=e; targets=t;}
9     public JUMP(Label target) {
10        this(new NAME(target), new LabelList(target,null));
11    }

```

2.3. MIPS 스택 프레임 레이아웃 검증 결과

위 패치들을 적용하여 Mips.FrameDriver를 단독 빌드 및 실행한 결과, MIPS 호출 규약에 부합하게 매개변수와 지역 변수가 할당되는 것을 확인하였다.

```

PS C:\Users\user\minijava_compiler> cd chap11
PS C:\Users\user\minijava_compiler\chap11> javac -encoding UTF-8 -d bin Mips/FrameDriver.java
PS C:\Users\user\minijava_compiler\chap11> java -cp bin Mips.FrameDriver
=== <=K args ===
# name: foo
# frameSize: 20
# formals:
arg[0]: InFrame offset=-12 (in-frame(-))
arg[1]: InReg
arg[2]: InFrame offset=-16 (in-frame(-))
=== >K args ===
# name: bar
# frameSize: 20
# formals:
arg[0]: InFrame offset=-12 (in-frame(-))
arg[1]: InReg
arg[2]: InFrame offset=-16 (in-frame(-))
arg[3]: InReg
arg[4]: InFrame offset=8 (stack-arg(+))
arg[5]: InFrame offset=12 (stack-arg(+))

```

- 매개변수가 4개 이하인 경우 (foo): 레지스터 할당 범위 내의 매개변수는 레지스터(InReg)에 매핑되고, 탈출(escape)하는 매개변수들은 프레임 내부 음수 오프셋(offset=-12, offset=-16)에 순차

배치됨을 검증함. (지역 변수는 이어서 음수 오프셋에 누적 할당됨)

- 매개변수가 4개를 초과하는 경우 (bar): 5번째 매개변수부터는 호출자 영역인 프레임 외부 양수 오프셋(offset=8, offset=12 등)에 스택 매개변수로 안전하게 매핑됨을 확인함.

2.4. 의미 분석 회귀 검증 시도 및 빌드 에러 해결

프레임 레이아웃 검증(FrameDriver)을 마친 후, 중간 표현 번역(7장)으로 진입하기 전, 5장의 의미 분석기(Type Checker)가 단일 폴더 통합 과정에서 깨지지 않았는지 회귀 테스트를 먼저 수행하고자 하였다.

그러나 전체 소스코드를 재귀적으로 일괄 컴파일하는 과정에서 뜻밖의 대량의 컴파일 에러가 추가로 발생하였다.

```
C:\Users\user\minijava_compiler\chap11\Canon\StmListList.java:6: error: unclosed character literal
<script type="text/javascript">window.addEventListener('DOMContentLoaded',function(){var v=archive_analytics.values;v.service='wb';v.
server_name='wwwb-app217.us.archive.org';v.server_ms=287;archive_analytics.send_pageview({});});</script>
                                                                    ^
C:\Users\user\minijava_compiler\chap11\Canon\StmListList.java:6: error: unclosed character literal
<script type="text/javascript">window.addEventListener('DOMContentLoaded',function(){var v=archive_analytics.values;v.service='wb';v.
server_name='wwwb-app217.us.archive.org';v.server_ms=287;archive_analytics.send_pageview({});});</script>
                                                                    ^
C:\Users\user\minijava_compiler\chap11\Canon\StmListList.java:6: error: unclosed character literal
<script type="text/javascript">window.addEventListener('DOMContentLoaded',function(){var v=archive_analytics.values;v.service='wb';v.
server_name='wwwb-app217.us.archive.org';v.server_ms=287;archive_analytics.send_pageview({});});</script>
                                                                    ^
```

에러 1: Canon/StmListList.java 웹 마크업 깨짐 버그

- 원인: 교재 원본 뼈대 코드 중 Canon/StmListList.java 파일이 HTML 배너 및 스크립트가 코드 내부에 그대로 삽입되어, 자바 컴파일러가 클래스 정의 대신 HTML 태그를 읽어 97개의 컴파일 구문 에러를 뱉음.

```
minijava_compiler > chap11 > Canon > StmListList.java
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <title>Wayback Machine</title>
5  <script src="//archive.org/includes/athena.js" type="text/javascript"></script>
6  <script type="text/javascript">window.addEventListener('DOMContentLoaded',function(){var v=archive_analytics.values;v.service='wb';v.
7  <script type="text/javascript" src="https://web-static.archive.org/_static/js/jquery-1.11.1.min.js"></script>
8  <link rel="stylesheet" type="text/css" href="https://web-static.archive.org/_static/css/wayback.css">
9  <link rel="stylesheet" type="text/css" href="https://web-static.archive.org/_static/css/wayback.css">
10 <script src="https://web-static.archive.org/_static/js/jquery-1.11.1.min.js"></script>
11 </head>
12 <body style="height:100vh;overflow:hidden;margin:0;display:flex;flex-flow:row;">
```

- 해결: 깨진 HTML 배너 영역을 모두 들어내고, StmListList 클래스의 원래 자바 소스코드로 복원하여 일괄 컴파일 오류를 해결함.

```
chap11 > Canon > StmListList.java
1  package Canon;
2
3  public class StmListList {
4      public Tree.StmList head;
5      public StmListList tail;
6      public StmListList(Tree.StmList h, StmListList t) {
7          head = h;
8          tail = t;
9      }
10 }
```

예러 2: Windows 파일 시스템의 파일명 대소문자 충돌 (EXP vs Exp)

- 원인: Windows의 기본 파일시스템(NTFS)은 파일명의 대소문자를 구분하지 않는다(Case-insensitive). 이로 인해 문장(Statement)을 나타내는 Tree/EXP.java와 식(Expression)을 나타내는 Tree/Exp.java가 동시에 존재할 때, 컴파일러가 생성하는 EXP.class와 Exp.class 중 나중에 빌드되는 파일이 이전 파일을 덮어써 파일이 소실되는 버그가 발생했다. 결과적으로 런타임에 클래스 로더가 타입을 찾지 못해 java.lang.NoClassDefFoundError: Tree/Exp (wrong name: Tree/EXP) 예외가 발생하며 비정상 종료되었다.

```
1 package Tree;
2 import Temp.Temp;
3 import Temp.Label;
4 public class EXP extends Stm {
5     public Exp exp;
6     public EXP(Exp e) {exp=e;}
```

- 해결: 문장 식을 상징하는 EXP.java 클래스를 EXP_stm.java로 리네임하고, 식의 최상위 추상 클래스인 Exp1.java를 Exp.java로 개명한 뒤 패키지 외부 참조가 가능하도록 public 접근 제어자를 부여하였다. 이후 Print.java와 Canon.java에서 Tree.EXP를 가리키던 호출부들을 모두 Tree.EXP_stm으로 일괄 치환하여 윈도우 OS 내 파일 충돌 문제를 완벽히 우회하였다.

```
chap11 > Tree > EXP_stm.java
1 package Tree;
2 import Temp.Temp;
3 import Temp.Label;
4 public class EXP_stm extends Stm {
5     public Exp exp;
```

3.5. 의미 분석 회귀 검증 결과

위 컴파일 오류들을 수정한 최종 격리 환경에서 Main.java를 일괄 빌드 및 실행한 결과, 5장에서 구현한 의미 분석의 핵심 데이터 흐름과 결함 탐지 로직이 정상적으로 유지되고 있음을 확인하였다.

```
PS C:\Users\user\minijava_compiler\chap11> java -cp bin Main ../programs/TestDuplicate.java
class TestDuplicate {
    public static void main (String [] a) {
        System.out.println(new Fac().ComputeFac(10)); }
}

class Fac {
    int num;
    int num;
    public int ComputeFac (int num) {
        int num_aux;
        int num_aux;
        int num;
        return num_aux;
    }
    public int ComputeFac (int num) {
        return 0;
    }
}

class Fac {
}
TestDuplicate.java:11:5: error: variable num is already defined in class Fac
TestDuplicate.java:15:9: error: variable num_aux is already defined in method ComputeFac(int)
TestDuplicate.java:17:9: error: variable num is already defined in method ComputeFac(int)
TestDuplicate.java:22:5: error: method ComputeFac is already defined in class Fac
TestDuplicate.java:28:1: error: class Fac is already defined
Type check failed for ../programs/TestDuplicate.java
TEST PASS: TestDuplicate.java successfully generated exactly 5 errors.
```

3. 중간 표현(IR) 번역 및 제어 흐름 정형화(Chap 7, 8)

3.1. 중간 표현 번역기 구현

챕터 6의 검증된 프레임 구조와 타입 체커를 기반으로, AST 노드를 중간 표현(Tree IR)인 Tree.Stm과 Tree.Exp로 일대일 변환하는 visitor/IRTranslator.java를 설계 및 구현하였다.

이 번역 모듈의 상세 구현 내용 및 설계 메커니즘은 다음과 같다.

1. 클래스 상속 구조 및 필드 오프셋 계산 설계 (preScanClassLayouts & getOrBuildFields)

MiniJava는 필드 상속을 지원하므로 멤버 변수 오프셋을 구하기 위해서는 조상 클래스의 멤버 변수 크기 정보가 필수적이다. 이를 위해 AST 순회 전, 클래스 선언 정보를 선방문하여 상속 관계 및 메서드 시그니처, 그리고 멤버 필드 배치 오프셋을 MIPS 32비트 워드 크기(WORD_SIZE = 4)에 맞춰 정렬하고 캐싱하도록 구현했다.

동작 상세 메커니즘:

1. 클래스 메타데이터 수집: Program 노드를 만나면 preScanClassLayouts가 실행되어 프로그램 내 모든 클래스 선언(ClassDecl)을 클래스 선언 매핑에 등록하고, ClassDeclExtends에서 획득한 부모-자식 상속 관계를 부모 관계 정보에 캐싱한다.
2. 재귀적 상속 계층 추적: getOrBuildFields 메서드는 Memoization을 적용하여 배치 완료된 필드는 즉시 재사용하되, 새로운 자식 클래스일 경우 부모 클래스를 향해 재귀 호출을 수행한다. 이로 인해 부모의 필드 목록이 자식 클래스 필드 리스트의 맨 처음에 누적되도록 물리적으로 강제한다.
3. 오프셋 맵 구축: 완성된 누적 필드 목록을 순회하며 인덱스 i에 대해 $i * \text{WORD_SIZE}$ (4바이트) 단위로 상대 주소 오프셋을 계산하여 fieldOffsets에 기록한다. 이를 통해 부모로부터 상속받은 변수의 오프셋 호환성을 정상적으로 유지한다.

```
69     private ArrayList<String> getOrBuildFields(String cname, HashMap<String, syntaxtree.ClassDecl> classDecls) {
70         if (classFields.containsKey(cname)) {
71             return classFields.get(cname);
72         }
73         ArrayList<String> fields = new ArrayList<>();
74         String parent = classParents.get(cname);
75         if (parent != null) {
76             fields.addAll(getOrBuildFields(parent, classDecls));
77         }
78
79         syntaxtree.ClassDecl cd = classDecls.get(cname);
80         if (cd != null) {
81             syntaxtree.VarDeclList vd1 = null;
82             if (cd instanceof syntaxtree.ClassDeclSimple) {
83                 vd1 = ((syntaxtree.ClassDeclSimple) cd).v1;
84             } else if (cd instanceof syntaxtree.ClassDeclExtends) {
85                 vd1 = ((syntaxtree.ClassDeclExtends) cd).v1;
86             }
87         }
88     }
```

```

87         if (vd1 != null) {
88             for (int i = 0; i < vd1.size(); i++) {
89                 String fname = vd1.elementAt(i).i.s;
90                 fields.add(fname);
91                 String ftype = typeNameOf(vd1.elementAt(i).t);
92                 if (!classAllFieldTypes.containsKey(cname)) {
93                     classAllFieldTypes.put(cname, new HashMap<>());
94                 }
95                 classAllFieldTypes.get(cname).put(fname, ftype);
96             }
97         }
98     }
99
100     if (parent != null) {
101         HashMap<String, String> parentTypes = classAllFieldTypes.get(parent);
102         if (parentTypes != null) {
103             if (!classAllFieldTypes.containsKey(cname)) {
104                 classAllFieldTypes.put(cname, new HashMap<>());
105             }
106             classAllFieldTypes.get(cname).putAll(parentTypes);
107         }
108     }
109
110     classFields.put(cname, fields);
111
112     HashMap<String, Integer> offsets = new HashMap<>();
113     for (int i = 0; i < fields.size(); i++) {
114         offsets.put(fields.get(i), i * WORD_SIZE);
115     }
116     fieldOffsets.put(cname, offsets);
117
118     return fields;
119 }

```

2. 로컬 및 멤버 변수의 스코프 식별 설계 (varExp & assignVar)

프로그램 내부에서 변수를 참조하거나 값을 대입할 때, 번역기는 이 변수가 로컬 스택에 존재하는지 혹은 객체 힙 영역에 존재하는 필드인지 동적으로 스코프를 조회하여 주소를 매핑한다.

동작 상세 메커니즘:

1. 로컬 scope 검색: 현재 메서드의 바인딩 테이블인 환경(Environment) 테이블을 확인하여 매핑된 가상 레지스터 Temp가 존재하면 로컬 변수(또는 매개변수)로 인지하고 new Tree.TEMP(t) 주소 식을 바로 반환한다.
2. 힙 멤버 필드 검색: 로컬 테이블에 변수가 존재하지 않는다면, 현재 번역 중인 클래스의 fieldOffsets를 조회한다. 필드가 확인되면 암묵적으로 전달된 객체 참조 레지스터인 this 가상 레지스터를 호출하고, 멤버 변수 오프셋을 더해 new Tree.MEM(new Tree.BINOP(Tree.BINOP.PLUS, TEMP(this), CONST(offset))) 형태의 간접 주소 참조 문장을 방출한다.

```

167 private Tree.Exp varExp(String name) {
168     Temp t = env().get(name);
169     if (t != null) {
170         return new Tree.TEMP(t);
171     }
172
173     if (currentClassName != null) {
174         HashMap<String, Integer> offsets = fieldOffsets.get(currentClassName);
175         if (offsets != null && offsets.containsKey(name)) {
176             int off = offsets.get(name);
177             Temp thisTemp = env().get("this");
178             if (thisTemp != null) {
179                 return new Tree.MEM(bin(Tree.BINOP.PLUS, new Tree.TEMP(thisTemp), new Tree.CONST(off)));
180             }
181         }
182     }
183
184     return new Tree.TEMP(tempOf(name));
185 }

```


3. 객체 및 배열 동적 메모리 할당 설계 (visit(NewObject), visit(NewArray))

동적으로 인스턴스나 배열을 할당받는 문장을 번역할 때, MIPS 런타임 시스템 내부의 heap 할당 라이브러리 함수(MIPS 어셈블리로 바인딩된 _halloc)를 결합하여 물리 힙 주소를 획득하고 데이터를 초기화하도록 처리했다.

동작 상세 메커니즘:

1. NewObject: 해당 클래스의 총 필드 개수(fcount)에 4바이트를 곱한 크기(fcount * WORD_SIZE)를 산출하고 힙 할당 함수를 호출한다. 할당이 완료되어 힙 시작 주소가 반환되면, 해당 인스턴스의 안정성을 위해 필드 영역 전체를 상수 0으로 순차 대입 (MOVE(MEM(...), 0))하여 0 초기화 (Zero-initialization)하는 초기화 시퀀스를 사슬로 엮어 방출한다.
2. NewArray: 배열 크기식 e를 방문하여 배열 요소 개수 size를 획득하고, 배열 길이 저장용 헤더(4바이트)를 포함해 메모리를 할당하는 배열 할당 함수를 호출한다. 배열의 0번 오프셋 자리에 배열 길이 정보를 명시적으로 write하여 향후 length 명령 시 MEM 조회가 가능하도록 설계했다.

```
582 public void visit(syntaxtree.NewObject n) {
583     String cname = n.i.s;
584     int fcount = classFields.getDefault(cname, new ArrayList<>()).size();
585     int sizeBytes = fcount * WORD_SIZE;
586     Temp t = new Temp();
587     List<Tree.Stm> inits = new ArrayList<>();
588
589     inits.add(new Tree.MOVE(new Tree.TEMP(t), new Tree.CALL(new Tree.NAME(new Label(Mips.Frame.heapAllocLabel())), new Tree.ExplList(new Tree.CONST(sizeBytes), null))));
590
591     for (int i = 0; i < fcount; i++) {
592         int off = i * WORD_SIZE;
593         inits.add(new Tree.MOVE(new Tree.MEM(bin(Tree.BINOP.PLUS, new Tree.TEMP(t), new Tree.CONST(off))), new Tree.CONST(0)));
594     }
595
596     resultExp = new Tree.ESEQ(seq(inits), new Tree.TEMP(t));
597 }
```

4. 메서드 호출 및 동적 디스패치(Dynamic Dispatch) 설계 (visit(Call))

MiniJava의 다형성을 지원하기 위해, 컴파일러가 수신자 객체의 런타임 타입(Class tag)을 식별하고 적절한 타겟 메서드로 제어 흐름을 동적 분기시키는 동적 디스패치(Dynamic Dispatch) 번역로직을 설계 및 구현하였다.

동작 상세 메커니즘:

1. 클래스 후보군 해결 및 라벨 테이블 구축:
 - 호출 대상 메서드가 오버라이딩되어 있을 가능성이 있는 클래스 상속 관계를 역조사하여, 해당 메서드가 구현되어 있을 후보 클래스명들과 그에 대응하는 정적 메서드 엔트리 레이블 목록을 resolvedLabels 맵으로 추출한다.

```

588     public void visit(syntaxtree.Call n) {
589         String recvClass = receiverClassOf(n.e);
590         n.e.accept(this);
591         Tree.Exp recv = resultExp;
592
593         Tree.Explist args = new Tree.Explist(recv, null);
594         for (int i = 0; i < n.el.size(); i++) {
595             n.el.elementAt(i).accept(this);
596             args = append(args, resultExp);
597         }
598
599         if (recvClass != null) {
600             Map<String, String> resolvedLabels = new HashMap<>();
601             for (String cname : classTags.keySet()) {
602                 if (isSubclassOf(cname, recvClass)) {
603                     String declarer = findMethodDeclarer(cname, n.i.s, classDecls);
604                     if (declarer != null) {
605                         resolvedLabels.put(cname, methodLabel(declarer, n.i.s));
606                     }
607                 }
608             }

```

2. 수신자 주소 보존 및 클래스 태그 로드:

- 수신자 식 e가 중복 평가되어 부작용을 낳는 것을 방지하기 위해, 수신자 객체의 평가 결과를 임시 가상 레지스터(tRecv)에 한 번 대입(MOVE)하여 고정한다.
- 힙 메모리의 0번 오프셋(수신자 주소 영역의 첫 4바이트)을 역참조(new Tree.MEM(new Tree.TEMP(tRecv)))하여 런타임에 인스턴스에 탑재되어 있는 클래스 태그를 로드하고, 이를 가상 레지스터 tTag에 할당한다.

```

614     } else {
615         Temp tRecv = new Temp();
616         Tree.Stm saveRecv = new Tree.MOVE(new Tree.TEMP(tRecv), recv);
617         Temp tTag = new Temp();
618         Tree.Stm loadTag = new Tree.MOVE(new Tree.TEMP(tTag), new Tree.MEM(new Tree.TEMP(tRecv)));
619         Temp tRet = new Temp();
620         Label lEnd = new Label();
621
622         List<Tree.Stm> stms = new ArrayList<>();
623         stms.add(saveRecv);
624         stms.add(loadTag);

```

3. 조건부 분기 체인 구축

- resolvedLabels에 수집된 후보 클래스 정보들을 순회하며 각 클래스의 고유 클래스 태그(Tag ID)를 가져온다.
- tTag 레지스터에 로드된 런타임 클래스 태그 값과 후보 클래스 태그 상수가 일치하는지 비교하는 조건부 분기문(CJUMPEQ)을 생성한다.
- 태그가 일치할 경우(IMatch 레이블): MIPS 프레임 규약에 따라 수신자 임시 레지스터(tRecv)를 첫 번째 인자(implicit this)로 삽입하고, 나머지 인수들을 결합하여 Tree.CALL 문을 수행한다. 호출 반환값은 결과 임시 레지스터(tRet)에 저장(MOVE)하고, 다형성 분기 체인의 끝인 수렴 레이블(lEnd)로 무조건 점프(JUMP)한다.
- 태그가 불일치할 경우(INext 레이블): 다음 후보 클래스 태그와의 비교 조건부 분기로 흘러가도록 체인을 엮는다.

```

626  for (Map.Entry<String, String> entry : resolvedLabels.entrySet()) {
627      String cname = entry.getKey();
628      String label = entry.getValue();
629      int tag = classTags.get(cname);
630
631      Label lMatch = new Label();
632      Label lNext = new Label();
633
634      stms.add(new Tree.CJUMP(Tree.CJUMP.EQ, new Tree.TEMP(tTag), new Tree.CONST(tag), lMatch, lNext));
635      stms.add(new Tree.LABEL(lMatch));
636      Tree.Explist dispatchArgs = new Tree.Explist(new Tree.TEMP(tRecv), args.tail);
637      stms.add(new Tree.MOVE(new Tree.TEMP(tRet), new Tree.CALL(new Tree.NAME(new Label(label)), dispatchArgs)));
638      stms.add(new Tree.JUMP(lEnd));
639      stms.add(new Tree.LABEL(lNext));
640  }

```

4. 통합 반환 구조 패키징

- 체인의 맨 마지막에는 어떠한 태그 매칭에도 해당하지 않는 경우를 처리하는 Fallback 기본 메서드 호출을 위치시킨다.
- 이 복잡한 조건문 비교와 CALL 호출로 엮힌 문장(Stm)들의 연쇄 시퀀스를 ESEQ(StmList, Exp) 구조로 감싸고, 최종 평가 결과로서 결과 저장 가상 레지스터 식인 TEMP(tRet)을 반환함으로써, 미들엔드 및 백엔드가 해당 호출문을 단일한 표현식(Exp) 트리로 안전하게 평가할 수 있도록 보장하였다.

```

642  Tree.Explist dispatchArgs = new Tree.Explist(new Tree.TEMP(tRecv), args.tail);
643  stms.add(new Tree.MOVE(new Tree.TEMP(tRet), new Tree.CALL(new Tree.NAME(new Label(methodLabel)), dispatchArgs)));
644  stms.add(new Tree.LABEL(lEnd));
645
646  resultExp = new Tree.ESEQ(seq(stms), new Tree.TEMP(tRet));

```

3.2 중간 표현 번역 검증 결과

Main.java에 새로 설계한 7장 번역기(visitor.IRTranslator)를 결합하고, 샘플 프로그램인 programs/Factorial.java를 번역하여 MIPS 프레임 규약에 맞는 중간 표현(Tree IR)이 정상적으로 형성되는지 확인하였다.

Main.java 연동: 타입 체커가 성공하면 IRTranslator 인스턴스를 생성하고 translate(root)를 호출하여 프로서저 정보 리스트를 획득한 후, Tree.Print를 통해 가상 레지스터가 할당된 원시 IR Tree 문장을 출력하도록 구성함.

```

21  if (success) {
22      System.out.println("Successfully type checked for " + path);
23
24      // Chapter 7 IR Tree Translation
25      visitor.IRTranslator translator = new visitor.IRTranslator();
26      java.util.List<visitor.IRTranslator.ProcedureIR> procedures = translator.translate(root);
27      System.out.println("--- Chapter 7 IR Translation Result for " + path + " ---");
28      Tree.Print printIR = new Tree.Print(System.out);
29  for (visitor.IRTranslator.ProcedureIR proc : procedures) {
30      System.out.println("Procedure: " + proc.name);
31      printIR.prStm(proc.body);
32  }
33  } else {
34      System.err.println("Type check failed for " + path);

```

컴파일 및 실행 결과

사용 명령어:

```
$sources = Get-ChildItem -Recurse *.java | Select-Object -ExpandProperty FullName
```

```
javac -encoding UTF-8 -d bin $sources
```

```
java -cp bin Main ../programs/Factorial.java
```

```
Successfully type checked for ../programs/Factorial.java
--- Chapter 7 IR Translation Result for ../programs/Factorial.java ---
Procedure: main
SEQ(
  SEQ(
    SEQ(
      SEQ(
        SEQ(
          SEQ(
            SEQ(
              MOVE(
                TEMP t29,
                TEMP t11),
              MOVE(
                TEMP t30,
                TEMP t12)),
            MOVE(
              TEMP t31,
              TEMP t13)),
          MOVE(
            TEMP t32,
            TEMP t14)),
        MOVE(
          TEMP t33,
          TEMP t15)),
      MOVE(
        TEMP t34,
        TEMP t16)),
    MOVE(
      TEMP t35,
      TEMP t17)),
  MOVE(
    TEMP t36,
    TEMP t18)),
EXP_stm(
  CALL(
    NAME _print,
    CALL(
      NAME Fac_ComputeFac,
      ESEQ(
        MOVE(
          TEMP t0,
          CALL(
            NAME _heapAlloc,
            CONST 0)),
        TEMP t0),
      CONST 10))))),
```

타입 체커 통과 후 Factorial.java의 main 엔트리와 ComputeFac 프로시저가 MIPS 호출 규약에 맞게 정상적으로 변환됨을 확인하였다. 구체적으로, 프로시저 진입 및 탈출부에서 Callee-saves 레지

스터(s0~s7) 값을 가상 레지스터에 대피시켰다 복원하는 MOVE 시퀀스가 안전하게 생성되었으며, 메서드 호출(ComputeFac(10)) 시 수신자 객체 포인터(this)가 첫 번째 매개변수로 전달되고 이항 연산 인수(CONST 10)가 순서대로 바인딩되어 정적 레이블(Fac_ComputeFac)로 정상 매핑되었다. 또한 재귀 호출 과정에서도 객체 포인터를 유실 없이 추적하며 곱셈(MUL) 및 조건부 제어 흐름 분기(CJUMP)가 정상적으로 표현되었다.

3.3. Canonicalize 및 제어 흐름 linearize (Chap 8)

챕터 7에서 번역된 Tree IR은 중첩된 ESEQ나 임의의 순서로 실행되는 CALL 등을 포함하고 있어, 기계어로 일대일 매핑하기에는 구조가 지나치게 복잡하다. 이를 해결하기 위해 부작용을 제거하고, 트리 표현식을 linearize된 문장 리스트로 평탄화하며, 최적화된 조건부 분기 흐름을 가지도록 재배열하는 3단계 Canonicalize 모듈을 연동하였다. (교수님 제공 소스 코드 활용)

동작 상세 메커니즘 (Canon 3단계 파이프라인)

1. Tree.StmList linearize (Canon.Canon.linearize): 입력받은 IR Tree에서 모든 ESEQ를 제거한다. ESEQ(s, e)가 발견되면 문장 s와 식 e를 물리적으로 분리하고, 평가 순서를 보장하면서 부작용이 배제된 순수한 표현식 트리로 재구성한다. 최종적으로 ESEQ가 완전히 제거된 플랫한 Tree.StmList가 생성된다.
2. 기본 블록 생성 (Canon.BasicBlocks): linearize된 문장 리스트를 기본 블록 단위로 분할한다. 각 기본 블록은 반드시 하나의 LABEL로 시작하여 JUMP나 CJUMP로만 끝나는 제어 흐름 단위를 가지며, 블록 내부에는 라벨이나 점프문이 존재하지 않아 단일 진입 단일 진출 특성을 보장한다. 흐름의 끝에는 안전한 종료를 보장하는 종료 레이블을 배치한다.
3. 트레이스 스케줄링 (Canon.TraceSchedule): 생성된 기본 블록들을 순차적 실행 경로인 트레이스로 스케줄링하여 무조건 점프(JUMP) 명령을 최소화하고 조건부 분기(CJUMP)를 효율화한다. 특히 MIPS 아키텍처에서는 조건부 점프(CJUMP) 바로 다음에 거짓일 때 실행할 라벨이 물리적으로 위치하면 명령어가 단순화된다. TraceSchedule은 CJUMP의 false 라벨이 바로 뒤에 배치되도록 블록 순서를 최적화하고, 불필요한 점프문들을 제거하여 linearize된 어셈블리 명령어 후보 시퀀스로 재배열한다.

Main.java 연동:

```

21         if (success) {
22             System.out.println("Successfully type checked for " + path);
23
24             // Chapter 7 IR Tree Translation
25             visitor.IRTranslator translator = new visitor.IRTranslator();
26             java.util.List<visitor.IRTranslator.ProcedureIR> procedures = translator.translate(root);
27
28             // Chapter 8 Canon & Chapter 9 Instruction Selection (Codegen)
29             for (visitor.IRTranslator.ProcedureIR proc : procedures) {
30                 Tree.StmList linearized = Canon.Canon.linearize(proc.body);
31                 Canon.BasicBlocks blocks = new Canon.BasicBlocks(linearized);
32                 Canon.TraceSchedule trace = new Canon.TraceSchedule(blocks);

```

컴파일 및 실행결과

사용 명령어:

```
$sources = Get-ChildItem -Recurse *.java | Select-Object -ExpandProperty FullName
```

```
javac -encoding UTF-8 -d bin $sources
```

```
java -cp bin Main ../programs/Factorial.java
```

```
Procedure: main
LABEL L7
MOVE(
  TEMP t29,
  TEMP t11)
MOVE(
  TEMP t30,
  TEMP t12)
MOVE(
  TEMP t31,
  TEMP t13)
MOVE(
  TEMP t32,
  TEMP t14)
MOVE(
  TEMP t33,
  TEMP t15)
MOVE(
  TEMP t34,
  TEMP t16)
MOVE(
  TEMP t35,
  TEMP t17)
MOVE(
  TEMP t36,
  TEMP t18)
MOVE(
  TEMP t0,
  CALL(
    NAME _heapAlloc,
    CONST 0))
```

7장 결과와 달리, 식 평가 중간에 존재하던 중첩 ESEQ 노드들이 완전히 제거되어 linearize된 문장 리스트로 변환되었음을 확인하였다. 또한, 조건부 분기문(CJUMP) 바로 다음에 거짓(false) 타겟 라벨이 위치하도록 트레이스가 최적화 배치되어 거짓 분기 시의 무조건 점프(JUMP) 명령어가 제거되고 효율적인 Fall-through 구조로 제어 흐름이 정돈된 것을 확인하였다.

4. 명령어 선택 및 MIPS 기계어 코드 생성 (Chap 9)

4.1. 명령어 선택기 구현 및 타일 매칭 설계

챕터 8의 Canonicalize 단계를 거쳐 ESEQ가 배제되고 제어 흐름이 정리된 linearize된 문장 리스트(Tree.StmList)를 MIPS 32비트 어셈블리 명령어 리스트(Assem.InstrList)로 번역하기 위해, 트리 패턴을 하향식으로 최대 매칭하는 Maximal Munch 알고리즘 기반의 코드 생성 모듈(Codegen.java)을 설계 및 구현하였다.

1. 지시어 방출 및 백엔드 파이프라인 진입 구조 설계

```
9  public class Codegen {
10     private InstrList  ilist = null, last = null;
11     private final Frame frame;
12
13     public Codegen(Frame f) {
14         frame = f;
15     }
16
17     private void emit(Instr inst) {
18         if (last != null) {
19             last = last.tail = new InstrList(inst, null);
20         } else {
21             last = ilist = new InstrList(inst, null);
22         }
23     }
24
25     public InstrList codegen(StmList stms) {
26         ilist = last = null;
27         for (StmList l = stms; l != null; l = l.tail) {
28             munchStm(l.head);
29         }
30         return ilist;
31     }
}
```

Codegen 클래스는 생성자를 통해 Mips.Frame 객체를 주입받는다. codegen(StmList stms) 진입 메서드가 호출되면, 내부 방출 리스트의 시작 포인터 ilist와 끝 포인터 last를 초기화하고, 문장 리스트를 순회하며 munchStm(Stm)을 재귀 호출하여 매칭된 Assem.Instr 객체들을 emit 메서드를 통해 순차적으로 연결한다.

2. 기본 문장 타일링 설계 (munchStm)

```
--
33  private void munchStm(Stm s) {
34      if (s instanceof Tree.LABEL) {
35          Tree.LABEL l = (Tree.LABEL) s;
36          emit(new Assem.LABEL(l.label.toString() + ":\n", l.label));
37      } else if (s instanceof JUMP) {
38          JUMP j = (JUMP) s;
39          if (j.exp instanceof NAME) {
40              NAME n = (NAME) j.exp;
41              emit(new OPER("j `j0\n", null, null, new LabelList(n.label, null)));
42          } else {
43              Temp t = munchExp(j.exp);
44              emit(new OPER("jr `s0\n", null, new TempList(t, null)));
45          }
46      } else if (s instanceof CJUMP) {
47          CJUMP c = (CJUMP) s;
48          Temp a = munchExp(c.left);
49          Temp b = munchExp(c.right);
50          String op = branchOp(c.relop);
51          emit(new OPER(op + " `s0, `s1, `j0\n", null, new TempList(a, new TempList(b, null)), new LabelList(c.iftrue, null)));
52      } else if (s instanceof Tree.MOVE) {
53          Tree.MOVE m = (Tree.MOVE) s;
54          munchMove(m.dst, m.src);
55      } else if (s instanceof EXP_stm) {
56          EXP_stm e = (EXP_stm) s;
57          munchExp(e.exp);
58      } else {
59          throw new Error("Unsupported statement in Codegen: " + s.getClass());
60      }
61  }
```

문장(Stmt) 노드에 대해 최대 크기의 타일(Tile)을 재귀 매칭하고 명령어를 방출하도록 구현했다. LABEL은 MIPS 어셈블리 라벨로 매핑하고, JUMP는 정적 라벨(NAME)인 경우 j Label을, 임의의 수식일 경우 레지스터 간접 점프 jr \$reg를 생성한다. CJUMP는 비교 조건식 헬퍼(분기 조건 비교 연산)를 통해 MIPS 비교 분기문(beq, bne 등)으로 매핑한다.

3. 메모리 오프셋 연산 최적화 주소 매핑 (munchAddr)

```

214 private Addr munchAddr(Exp e) {
215     if (e instanceof BINOP) {
216         BINOP b = (BINOP) e;
217         if (b.binop == BINOP.PLUS) {
218             if (b.left instanceof CONST) {
219                 int k = ((CONST) b.left).value;
220                 return new Addr(munchExp(b.right), k);
221             } else if (b.right instanceof CONST) {
222                 int k = ((CONST) b.right).value;
223                 return new Addr(munchExp(b.left), k);
224             }
225         }
226     }
227     return new Addr(munchExp(e), 0);
228 }

```

MIPS의 간접 메모리 참조 규격 offset(base)을 활용해 불필요한 산술 덧셈 지시어의 낭비를 방지하도록 주소 계산 전담 헬퍼 메서드인 munchAddr를 설계했다. MEM(BINOP(PLUS, e1, CONST(k))) 혹은 MEM(BINOP(PLUS, CONST(k), e1))과 같이 메모리 참조 내부에 상수 가산 연산이 중첩된 형태를 먼저 탐지하여 베이스 레지스터 e1만 munchExp로 로드하고, 상수 k는 오프셋 변위값으로 직접 매핑한 Addr 객체를 반환하도록 구현했다.

4. 상수 가산 Immediate 연산 최적화 (munchExp)

```

102 if (b.binop == BINOP.PLUS && b.right instanceof CONST) {
103     Temp left = munchExp(b.left);
104     int val = ((CONST) b.right).value;
105     emit(new OPER("addi `d0, `s0, " + val + "\n", new TempList(t, null), new TempList(left, null)));
106 } else if (b.binop == BINOP.PLUS && b.left instanceof CONST) {
107     Temp right = munchExp(b.right);
108     int val = ((CONST) b.left).value;
109     emit(new OPER("addi `d0, `s0, " + val + "\n", new TempList(t, null), new TempList(right, null)));
110 } else {

```

수식(Exp) 중 BINOP(PLUS, left, CONST(val)) 또는 BINOP(PLUS, CONST(val), right) 형태를 탐지하면, 상수를 별도 레지스터에 로드(li)하는 비효율을 방지하고 MIPS Immediate 연산 명령인 addi \$dst, \$src, val 타일을 직접 매칭 방출하여 생성되는 총 명령어의 개수를 줄였다.

4.2. MIPS 함수 호출 및 매개변수 규약 준수 (munchCall)

함수 호출 식(CALL)을 기계어로 번역할 때는 MIPS Calling Convention을 엄격하게 준수하면서, 호출 이후 Liveness 분석과 레지스터 할당기의 안전성을 지키기 위해 매개변수 바인딩 루틴을 정교화하였다. 처음 4개의 인자는 매개변수 물리 레지스터인 \$a0 ~ \$a3으로 값 복사(move)되도록 생성하고, 5번째 인자부터는 스택에 저장한다. 특히 인자 전달 레지스터 \$a0 ~ \$a3을 정의된 레지스

터 목록에 명시적으로 누적 정의하여 함수 호출 시 해당 레지스터의 기존 활성 값이 덮어쓰워져 유실됨을 Liveness 분석기에 인지시켰다.

```
167 Templist callDefs = Frame.callerSaves();
168 callDefs = new Templist(Frame.RV2, callDefs);
169 callDefs = new Templist(Frame.RV, callDefs);
170 callDefs = new Templist(Frame.RA, callDefs);
171 callDefs = new Templist(Frame.argReg(0), callDefs);
172 callDefs = new Templist(Frame.argReg(1), callDefs);
173 callDefs = new Templist(Frame.argReg(2), callDefs);
174 callDefs = new Templist(Frame.argReg(3), callDefs);
175
```

4.3. 가상 레지스터 매핑

```
28 // Chapter 8 Canon & Chapter 9 Instruction Selection (Codegen)
29 for (visitor.IRTranslator.ProcedureIR proc : procedures) {
30     Tree.StmList linearized = Canon.Canon.linearize(proc.body);
31     Canon.BasicBlocks blocks = new Canon.BasicBlocks(linearized);
32     Canon.TraceSchedule trace = new Canon.TraceSchedule(blocks);
33
34     Codegen.Codegen codegen = new Codegen.Codegen(proc.frame);
35     Assem.InstrList instrs = codegen.codegen(trace.stms);
36
37     // Custom TempMap to show register names ($a0 etc.) or temporary numbers (t82 etc.) instead of null
38     Temp.TempMap customMap = new Temp.TempMap() {
39         public String tempMap(Temp.Temp t) {
40             String name = Mips.Frame.regNameMap().tempMap(t);
41             if (name != null) return name;
42             return t.toString();
43         }
44     };
45 }
```

아직 물리 레지스터 할당이 구현되기 전인 9장 백엔드 개발 시점에, 어셈블리 지시어 포매팅 시 가상 레지스터가 null로 지칭되는 문제를 해결하기 위해 하이브리드 TempMap인 임시 레지스터 매핑 테이블을 드라이버에 도입하였다. 이미 정해진 물리 레지스터(인자 전달 a0-a3 등)는 고정 기호로 출력하고, 미할당 가상 레지스터는 고유 꼬리표(t82 등)로 덤프한다.

4.4. 명령어 선택 검증 결과 (실행 결과 삽입 영역)

사용 명령어:

```
$sources = Get-ChildItem -Recurse *.java | Select-Object -ExpandProperty FullName
```

```
javac -encoding UTF-8 -d bin $sources
```

```
java -cp bin Main ../programs/Factorial.java
```

```

Successfully type checked for ../programs/Factorial.java
--- Chapter 9 MIPS Assembly for Procedure main ---
L7:
move t29, $s0
move t30, $s1
move t31, $s2
move t32, $s3
move t33, $s4
move t34, $s5
move t35, $s6
move t36, $s7
li t50, 0
move $a0, t50
jal _heapAlloc
move t0, $v0
li t51, 10
move $a0, t0
move $a1, t51
jal Fac_ComputeFac
move t49, $v0
move $a0, t49
jal _print
move t52, $v0
move $s7, t36
move $s6, t35
move $s5, t34
move $s4, t33
move $s3, t32
move $s2, t31
move $s1, t30
move $s0, t29
j L6
L6:
--- Chapter 9 MIPS Assembly for Procedure Fac_ComputeFac ---
L9:
move t37, $a0
move t38, $a1
move t41, $s0
move t42, $s1

```

가상 레지스터가 매핑된 MIPS 어셈블리 코드가 정상적으로 추출되었음을 확인할 수 있다. 구체적으로 main 및 ComputeFac 프로시저의 진입과 탈출부에 Callee-save 물리 레지스터(\$s0 ~ \$s7) 대피 및 복원을 위한 복사(move) 시퀀스가 올바르게 배치되었고, 메서드 호출 시 수신자 객체 포인터(t0)와 인수 값 10이 인자 전달 레지스터인 \$a0와 \$a1로 복사되어 전달되는 Calling Convention이 정밀하게 준수되었다. 아울러 뺄셈 및 곱셈 연산이 MIPS의 sub, mul 명령으로 적절하게 munching 되었고, num < 1 조건식이 MIPS 비교 분기문인 blt 지시어로 매핑되는 등 Maximal Munch 알고리즘에 기초한 최적의 명령어 선택 아키텍처가 정상 기능함을 확인하였다.

5. 활성 분석 및 흐름 그래프 설계 (Chap 10 - 옵션)

명령어 선택 단계를 거쳐 생성된 가상 어셈블리 명령어 스트림을 입력받아, 변수들의 유효 수명(Liveness)을 계산하고 물리 레지스터 배정의 핵심 충돌 단서가 되는 간섭 그래프(Interference Graph)를 구축하기 위해 활성 분석 모듈(FlowGraph 및 Liveness.java)을 결합하였다. (교수님 제공 소스 코드 활용)

동작 상세 메커니즘

1. 제어 흐름 그래프 구성 (FlowGraph/AssemFlowGraph.java):

- 가상 어셈블리 명령어 리스트(Assem.InstrList)의 각 인스트럭션을 노드로 매핑하여 제어 흐름 그래프를 모델링한다.
- 제어 흐름 엣지 연결: 분기 지시어(JUMP, CJUMP)인 경우, 타겟 점프 라벨 노드로의 분기 엣지를 형성한다. 무조건 점프(j, jr)를 수행하는 종단 노드(isTerminal)가 아닐 경우, 다음 명령어 노드로의 Fall-through 엣지를 자동으로 연결한다. 각 노드를 순회하며 해당 명령어가 정의(Write)하는 레지스터 목록 def(node)와 참조(Read)하는 레지스터 목록 use(node)를 추출한다.

2. 반복 데이터 흐름 방정식(FlowGraph/Liveness.java - analyze):

- 각 프로그램 포인트에서 변수들이 나중에 사용될 가능성이 있는 상태를 계산하기 위해, 아래의 데이터 흐름 방정식을 소스 코드가 안정 상태에 수렴할 때까지 반복하여 해결한다.
- $in[n] = use[n] \cup (out[n] - def[n])$ (노드 진입 시 활성 변수 = 노드 내 사용 변수와 노드 탈출 시 활성 변수 중 정의 변수를 제외한 변수들의 합집합)
- $out[n] = \bigcup \{ in[s] \} \text{ (s는 n의 후속 노드 succ[n])}$ (노드 탈출 시 활성 변수 = 모든 후속 노드의 진입 시 활성 변수들의 합집합)
- 활성 집합 연산의 정합성과 속도 향상을 위해 가상 레지스터 번호(t1, t2 등) 기준으로 자동 오름차순 정렬을 수행하는 정밀 매퍼(TreeSet 및 TEMP_COMPARATOR) 자료구조를 결합하여 연산을 수행한다.

3. 간섭 그래프 구축 (FlowGraph/Liveness.java - Interference.build):

- 활성 분석이 완료된 in/out 정보를 기반으로 충돌 관계를 추출하는 중첩 Interference 클래스를 구동한다.
- 간섭 엣지(Interference Edge) 생성: 임의의 노드 n에 대해, 해당 지점에서 값이 정의(Write)되는 임의의 레지스터 def[n] 집합에 속하는 레지스터 d와 해당 노드 탈출 시점에 활성 상태인 레지스터 out[n] 집합에 속하는 레지스터 l 사이에 무방향 간섭 엣지를 형성한다. (단, mov 명령인 경우 d와 소스 피연산자 $s \in use[n]$ 사이의 간섭은 배제하여 Coalescing의 가능성을 남긴다)

```

121     if (flow.isMove(n) && defList != null && useList != null) {
122         for (TemplList d = defList; d != null; d = d.tail) {
123             for (TemplList u = useList; u != null; u = u.tail) {
124                 moves = new MoveList(ensure(u.head), ensure(d.head), moves);
125             }
126         }
127     }
128 }

```

이 과정에서 발견되는 mov 명령어들의 소스-목적지 쌍들을 따로 묶어 MoveList로 캐싱해 둬서 후속 11단계의 George/Briggs Coalescing 최적화로 이송한다.

RegAlloc.java 연동

레지스터 할당기의 메인 루프 안에서 매 Spill 루프마다 그래프를 새롭게 동적 구축하기 위한 핵심 엔진으로 통합되었다.

```

44     final int maxIterations = 8;
45     for (int iter = 0; iter < maxIterations; iter++) {
46         FlowGraph fg = new AssemFlowGraph(current);
47         Liveness live = new Liveness(fg);
48         InterferenceGraph ig = live.interferenceGraph();
49
50         // Compute loop-aware spill weights
51         Map<Temp, Double> weights = computeSpillWeights(current);
52
53         Color color = new Color(ig, Frame.regNameMap(), regs, weights);
54         TemplList roundSpills = color.spills();

```

6. 레지스터 할당 및 그래프 채색 최적화 (Chap 11 - 옵션)

6.1. 레지스터 할당 구조 및 2단계 파이프라인 설계

명령어 선택기(Codegen)가 생성한 무한한 가상 레지스터(Temp)를 MIPS 아키텍처의 물리 레지스터로 매핑하기 위해 Graph Coloring 알고리즘 기반의 레지스터 할당기(RegAlloc.java) 및 채색기(Color.java)를 설계 및 구현하였다.

교재 표준 알고리즘은 Simplify, Coalesce, Freeze, Spill 단계를 실시간으로 교차 실행하므로 7개 이상의 상호 연계된 대기열 집합을 정밀 제어해야 하기 때문에 구현이 극도로 복잡하다.

본 컴파일러는 구현 복잡성을 줄이기 위해 Stage 1: Iterative Coalescing 단계와 Stage 2: Simplify & Spill 단계를 순차적으로 분리하는 2단계 분리형 파이프라인을 설계하여 적용하였다. 1단계에서 MIPS 아키텍처 범위 내에서 병합 가능한 move 명령어들을 완전히 한계치까지 수렴시켜 소거하므로, 2단계에서는 Coalescing 포기 여부를 결정하는 복잡한 Freeze 과정 없이도 정상적인 레지스터 병합 성능을 제공하는 견고하고 효율적인 구조를 완성하였다.

6.2. 기본 골격 코드와의 기능 대조

교수님이 제공해 주신 뼈대 파일(RegAlloc_FlowGraph)의 원본 소스와 본 컴파일러에 이식된 최종 최적화 소스를 대조 분석한 내역은 다음과 같다.

1. Coalescing 여부: 원본 코드는 Coalescing 루틴이 아예 존재하지 않아 다량의 무의미한 move 명령어가 기계어에 잔존했으나, 최종본은 George/Briggs 알고리즘과 Union-Find를 결합하여 Move 명령어를 원천 소거한다.
2. Spill Heuristics: 원본 코드는 임의로 첫 번째 비물리 노드를 선택하여 스푼시켰으나, 최종본은 루프 깊이에 비례해 가중치를 산정하는 Loop-Aware Spill Heuristics를 완비하여 루프 내부 레지스터를 영속 보호한다.
3. Spill Slot 할당 방식: 원본 코드는 가상 레지스터마다 고유 메모리를 격리했으나, 최종본은 Coalescing 관계에 있는 변수들이 동일 스택 주소를 포인팅하도록 Slot Sharing을 구현했다.
4. Spill 코드 안전성: 원본 코드의 lw/sw 지시어 템플릿 끝에 개행(wn)이 유실되어 명령어 텍스트가 합쳐져 MARS 시뮬레이터가 컴파일 에러를 뱉던 치명적 결함을 올바르게 해결했다.
5. 자기 복사 소거: 최종 컴파일러 기계어 방출 시점에 동일 물리 레지스터 간 복사(move \$s0, \$s0)를 필터링하여 소거하는 기법을 더했다.

6.3. 핵심 알고리즘 구현 상세

1. George 기준의 Conservative Coalescing 알고리즘

```

96  private boolean checkGeorge(CoalescedGraph cg, Node nonPre, Node pre, Set<Node> precolored, int K) {
97      for (Node t : cg.adj(nonPre)) {
98          if (precolored.contains(t)) continue;
99          if (cg.interferes(t, pre)) continue;
100         if (cg.degree(t) < K) continue;
101         return false;
102     }
103     return true;
104 }

```

코드 상세 설명:

비물리 레지스터(가상 레지스터) 일반 가상 노드와 물리 레지스터(Precolored) 노드 pre를 안전하게 하나로 합칠 수 있는지 판별한다. 일반 가상 노드 노드의 모든 이웃 노드 t를 순회하며 다음 세 가지 안전 조건 중 하나를 충족하는지 검사한다.

첫째, 이웃 노드 t가 이미 물리 레지스터인 경우(사전 컬러링 집합에 속한 경우), 추가적인 레지스터 간섭 제약을 형성하지 않으므로 즉시 통과시킨다.

둘째, 이웃 노드 t와 대상 물리 레지스터 pre가 이미 서로 간섭 엣지를 맺고 있는 경우(간섭 맺고 있는 경우), 두 레지스터가 합쳐지더라도 새로운 간섭이 유발되지 않으므로 통과시킨다.

셋째, 이웃 노드 t가 비물리 레지스터이지만 현재 차수가 가용 레지스터 개수인 K 미만인 경우(노드의 차수가 레지스터 개수 K보다 작은 경우), Simplify 단계에서 무조건 제거되어 색상을 배정받

을 수 있는 안전지대에 위치하므로 통과시킨다.

만약 위 세 가지 안전 조건 중 단 하나도 충족하지 못하는 이웃 노드 t 가 단 하나라도 존재한다면 Coalescing은 불가능하므로 false를 반환하고 기각한다.

2. Briggs 기준의 Conservative Coalescing 알고리즘

```
106  private boolean checkBriggs(CoalescedGraph cg, Node u, Node v, int K) {
107      int significantDegreeCount = 0;
108      Set<Node> unionNeighbors = new HashSet<>(cg.adj(u));
109      unionNeighbors.addAll(cg.adj(v));
110      for (Node t : unionNeighbors) {
111          if (cg.degree(t) >= K) {
112              significantDegreeCount++;
113          }
114      }
115      return significantDegreeCount < K;
116  }
```

코드 상세 설명:

가상 레지스터 노드 u 와 가상 레지스터 노드 v 를 하나의 대표 주소로 Coalescing할 수 있는지 검사한다.

두 노드가 Coalescing되었을 때 생겨날 단일 노드의 잠재적 이웃 노드 목록을 구하기 위해, u 와 v 의 인접 노드들의 합집합 집합인 합집합 이웃 노드들을 구성한다.

합집합 노드들을 순회하며 각 노드 t 중 현재 차수가 K 이상인 고차수 노드들의 개수(고차수 노드의 수)를 누적 카운트한다.

이 고차수 노드의 총합이 K 보다 작은 경우(고차수 노드의 수 $< K$), 두 레지스터를 합치더라도 최종 컬러링을 해치지 않고 무조건 Simplify가 가능함이 수학적으로 증명되므로 true를 반환하여 안전하게 가상 레지스터 간의 Coalescing을 승인한다.

3. 선행 coalescing 제어 루프

```
139  // 1. Coalescing Loop using George and Briggs
140  boolean coalescedAny;
141  do {
142      coalescedAny = false;
143      CoalescedGraph cg = new CoalescedGraph(allNodes, precolored, this);
144
145      for (MoveList ml = ig.moves(); ml != null; ml = ml.tail) {
146          Node u = getAlias(ml.src);
147          Node v = getAlias(ml.dst);
148
149          if (u == v) continue;
150          if (cg.interferes(u, v)) continue;
151
152          boolean uPre = precolored.contains(u);
153          boolean vPre = precolored.contains(v);
```

```

154
155     if (uPre && vPre) continue; // both precolored
156
157     boolean safe = false;
158     if (uPre || vPre) {
159         Node pre = uPre ? u : v;
160         Node nonPre = uPre ? v : u;
161         safe = checkGeorge(cg, nonPre, pre, precolored, K);
162         if (safe) {
163             aliasMap.put(nonPre, pre);
164         }
165     } else {
166         safe = checkBriggs(cg, u, v, K);
167         if (safe) {
168             aliasMap.put(v, u);
169         }
170     }
171
172     if (safe) {
173         coalescedAny = true;
174         break; // rebuild graph and scan again
175     }
176 }
177 } while (coalescedAny);

```

코드 상세 설명:

Simplify 루프에 들어가기 전 Coalescing 가능성을 한계점까지 유도하는 제어부이다. Coalescing 완료 여부 플래그를 두어 do-while 루프 구조로 구동된다.

매 루프의 시작 시점에 대표 노드 매핑 정보(aliasMap)를 바탕으로 최신의 관계를 반영한 임시 간섭 그래프(cg)를 동적으로 재생성한다.

그 후 모든 복사 명령 관계 리스트(ig.moves())를 순회하며, 소스와 목적지 가상 노드의 최종 대표 값 u와 v를 획득하여 동일 노드이거나 이미 다른 연산으로 인해 간섭(cg.interferes(u, v)) 중이면 Coalescing을 패스한다.

둘 다 물리 레지스터가 아닌 한에서, 물리 레지스터가 포함되어 있다면 George 기준 판정을, 둘 다 가상 레지스터라면 Briggs 기준 판정을 판정해 안전함이 확인되면 대표 노드 맵(aliasMap)에 관계를 등록하고 Coalescing 완료 여부 플래그를 세운 뒤, 즉시 내부 루프를 탈출(break)하여 간섭 그래프를 갱신하고 검사를 처음부터 재수행한다. 더 이상 안전하게 합칠 Move문이 없을 때 루프를 빠져나간다.

4. 루프 중첩 기반 가중치 산출 알고리즘

```

99 private Map<Temp, Double> computeSpillWeights(InstrList instrs) {
100     List<Assem.Instr> list = new ArrayList<>();
101     for (InstrList p = instrs; p != null; p = p.tail) {
102         list.add(p.head);
103     }
104
105     Map<Label, Integer> labelIndices = new HashMap<>();
106     for (int i = 0; i < list.size(); i++) {
107         Assem.Instr ins = list.get(i);
108         if (ins instanceof LABEL) {
109             labelIndices.put(((LABEL) ins).label, i);
110         }
111     }
112
113     List<int[]> loops = new ArrayList<>();
114     for (int i = 0; i < list.size(); i++) {
115         Assem.Instr ins = list.get(i);
116         Targets jt = ins.jumps();
117         if (jt != null && jt.labels != null) {
118             for (LabelList ll = jt.labels; ll != null; ll = ll.tail) {
119                 Label lbl = ll.head;
120                 if (labelIndices.containsKey(lbl)) {
121                     int targetIdx = labelIndices.get(lbl);
122                     if (targetIdx < i) {
123                         loops.add(new int[]{targetIdx, i});
124                     }
125                 }
126             }
127         }
128     }
129
130     int[] depth = new int[list.size()];
131     for (int i = 0; i < list.size(); i++) {
132         int d = 0;
133         for (int[] loop : loops) {
134             if (i >= loop[0] && i <= loop[1]) {
135                 d++;
136             }
137         }
138         depth[i] = d;
139     }
140
141     Map<Temp, Double> weights = new HashMap<>();
142     for (int i = 0; i < list.size(); i++) {
143         Assem.Instr ins = list.get(i);
144         int d = depth[i];
145         double occurrenceWeight = (d == 0) ? 1.0 : 10.0 * d;
146
147         TempList defs = ins.def();
148         for (TempList p = defs; p != null; p = p.tail) {
149             weights.put(p.head, weights.getOrDefault(p.head, 0.0) + occurrenceWeight);
150         }
151         TempList uses = ins.use();
152         for (TempList p = uses; p != null; p = p.tail) {
153             weights.put(p.head, weights.getOrDefault(p.head, 0.0) + occurrenceWeight);
154         }
155     }
156
157     return weights;
158 }

```

코드 상세 설명:

MIPS 어셈블리 지시어 스트림 전체를 정적 탐색하여 루프 깊이에 비례한 스푼 기피 가중치를 계산하는 최적화 모듈을 구현하였다.

먼저 역방향 분기를 스캔하기 위해 각 어셈블리 명령어의 점프 타겟 라벨을 전수 조사하며, 목적지 라벨의 정적 코드 위치가 현재 점프문 위치보다 앞선 인덱스를 가질 경우 이를 물리적인 역방향 분기 구조로 판정하여 루프 범위 경계를 획득한다.

그 후 전체 명령어 스트림을 순회하며 각 명령어가 앞서 수집된 루프 범위 영역들에 중첩하여 속해 있는지를 판별해 각 명령어 라인별 루프 중첩 깊이를 측정한다.

최종적으로 루프 깊이가 0인 일반 영역은 1.0의 기본 발생 가중치를 부여하지만, 루프 중첩이 발생한 영역의 명령어에는 깊이에 따라 10의 거듭제곱 형태로 강한 지수 가중치를 부여하도록 산출한다.

가상 레지스터가 정의되거나 사용되는 지점의 명령어 가중치를 가상 레지스터별 맵에 누적 산출함으로써 루프 내부 변수가 스푼되는 성향을 수학적으로 억제하고 물리 레지스터 할당 우선순위를 향상시켰다.

5. Simplify 및 스푼 결정 루틴

```

204 // Simplify phase
205 while (removed.size() < activeNodes.size()) {
206     Node pick = null;
207     if (!worklist.isEmpty()) {
208         pick = worklist.remove(worklist.size() - 1); // pop
209     } else {
210         // No Low-degree nodes: pick non-precolored, non-removed node with Lowest Loop-aware spill priority
211         double minPriority = Double.MAX_VALUE;
212         for (Node n : activeNodes) {
213             if (!removed.contains(n) && !precolored.contains(n)) {
214                 double w = weights.getOrDefault(ig.gtemp(n), 0.0);
215                 int deg = degree.getOrDefault(n, 0);
216                 double priority = w / (deg + 1); // avoid division by zero
217                 if (priority < minPriority) {
218                     minPriority = priority;
219                     pick = n;
220                 }
221             }
222         }
223         if (pick == null) {
224             break; // only precolored nodes remain
225         }
226     }
227     stack.push(pick);
228     removed.add(pick);
229
230     // Decrement neighbor degrees and add newly-Low-degree neighbors to worklist
231     for (Node m : cg.adj(pick)) {
232         if (!removed.contains(m)) {
233             int d = degree.getOrDefault(m, 0);
234             if (d > 0) degree.put(m, d - 1);
235             if (!precolored.contains(m) && degree.get(m) < K && !worklist.contains(m)) {
236                 worklist.add(m);
237             }
238         }
239     }
240 }
241

```

코드 상세 설명:

Coalescing 단계가 포화 수렴한 후 스택에 노드들을 담은 제어부이다. 모든 노드가 대피 제거될 때까지 작동한다.

차수가 K 미만인 저차수 노드 리스트가 빌드되어 있다면(`worklist.isEmpty() == false`), 즉시 노드를 대피 스택에 넣고 그래프에서 간접 제거 처리한다(Simplify).

만약 Simplify할 수 있는 안전한 저차수 노드가 더 이상 없다면(모든 활성 노드의 차수가 K 이상인 시점), 외부에서 주입받은 루프 깊이 가중치 맵(weights)과 현재 차수(degree)를 대조하여 각

노드 n 의 스페일 우선순위 수식인 $w / (\deg + 1)$ 값을 전수 계산한다. 이 우선순위 수치값이 가장 낮게 평가된 노드(루프 바깥에 위치하고 호출 빈도가 가장 낮은 변수)를 잠재적 스페일 노드로 낙점하여 대피 스택에 적재한다(Spill Heuristic).

노드가 스택에 들어갈 때마다 그 이웃 노드들의 임시 차수를 1씩 삭감하며, 차수가 K 미만으로 떨어진 이웃이 있다면 다시 worklist에 추가하여 Simplify 기회를 제공한다.

6. Coalescing된 스페일 슬롯 공유 및 주소 매핑

```
283 private void ensureSpillSlots(Frame f, Set<Temp> spilledSet, Color color) {
284     for (Temp t : spilledSet) {
285         Temp aliasT = color.getAliasTemp(t);
286         if (!spillOffsets.containsKey(aliasT)) {
287             Access a = f.allocLocal(true);
288             if (!(a instanceof InFrame)) {
289                 throw new RuntimeException("Expected InFrame for spilled temp");
290             }
291             spillOffsets.put(aliasT, ((InFrame) a).offset);
292         }
293         spillOffsets.put(t, spillOffsets.get(aliasT));
294     }
295 }
```

코드 상세 설명:

Graph Coloring 실패로 인해 최종 스페일 판정을 받은 가상 레지스터 변수들에 대해 활성 프레임 레이아웃 내에 상대 주소를 할당하는 구간이다.

스페일된 각 가상 레지스터 t 에 대해, Coalescing 관계를 조사하여 해당 변수의 최종 대표 노드(alias)를 획득한다.

만약 대표 노드가 아직 스택 프레임 주소를 할당받지 못했다면, MIPS 활성 프레임 객체의 로컬 변수 할당 함수를 구동해 고유의 프레임 음수 오프셋 주소를 획득 및 등록한다.

그 외 동일 대표 노드를 향하는 Coalescing 대상 레지스터 t 들에게는 앞서 대표 노드가 획득한 오프셋 주소를 동일하게 복사 바인딩한다. 이를 통해 Coalescing된 변수들이 스택 메모리 상에서도 올바르게 동일한 물리적 위치를 공유하게 조율하여 데이터 붕괴를 예방하고 불필요한 스택 영역의 낭비를 소거하였다.

7. 중복 복사 명령어 명령어 제거

```
47 for (Assem.InstrList il = procOutput.body; il != null; il = il.tail) {
48     // Eliminate redundant moves (move $s0, $s0)
49     if (il.head instanceof Assem.MOVE) {
50         Assem.MOVE m = (Assem.MOVE) il.head;
51         String dstName = regAlloc.tempMap(m.dst);
52         String srcName = regAlloc.tempMap(m.src);
53         if (dstName != null && dstName.equals(srcName)) {
54             continue;
55         }
56     }
57     System.out.print(il.head.format(regAlloc));
58 }
```

코드 상세 설명:

레지스터 할당이 완결된 MIPS 지시어 스트림을 순회하며 최종 어셈블리 문자열을 출력하는 메인 드라이버의 핵심 최적화 제어문이다.

지시어가 레지스터 이송 명령어(Assem.MOVE)에 해당하는 경우, 할당기 결과 맵(할당 결과 테이블)을 질의하여 해당 인스트럭션의 목적지 레지스터 목적지 레지스터와 소스 레지스터에 최종 매핑된 MIPS 물리 레지스터 이름을 각각 획득한다.

획득한 두 물리 레지스터 명칭이 정상적으로 일치한다면(최종 매핑된 두 레지스터 이름이 동일한 경우), 이는 MIPS 어셈블리 수준에서 의미가 없는 복사 연산(예: move \$s0, \$s0)이므로 어셈블리 텍스트 방출을 원천적으로 생략(continue) 처리하여 명령어 개수와 지연율을 최적화한다.

6.4. 레지스터 할당 결과 검증 및 MIPS 최종 출력

```
# Successfully type checked for ../programs/Factorial.java
main:
addiu $sp, $sp, -8
sw $ra, 4($sp)
sw $fp, 0($sp)
addiu $fp, $sp, 8

L7:
li $a0, 4
jal _heapAlloc
move $a0, $v0
li $v1, 1
sw $v1, 0($a0)
li $a1, 10
jal Fac_ComputeFac
move $a0, $v0
jal _print
j L6
L6:
li $v0, 10
syscall

Fac_ComputeFac:
addiu $sp, $sp, -12
sw $ra, 8($sp)
sw $fp, 4($sp)
addiu $fp, $sp, 12

L9:
sw $a1, -12($fp)
li $v1, 0
li $v0, 1
lw $t0, -12($fp)
blt $t0, $v0, L0
L1:
L2:
li $v0, 0
bne $v1, $v0, L3
L4:
lw $v1, -12($fp)
sw $v1, -12($fp)
li $v1, 1
```

출력된 실제 MIPS 코드를 분석하면, 가상 레지스터가 표준 MIPS 호출 규약에 부합하는 물리 레지스터(\$a0, \$a1, \$v0, \$v1, \$t0) 및 활성 스택 프레임 상대 주소로 정밀 변환되었음을 확인하였다.

main 함수 진입 시 스택 공간을 확보(addiu \$sp, \$sp, -8)하고 복귀 주소 \$ra 및 프레임 포인터 \$fp를 임시 백업한 후, 힙 할당 함수(_heapAlloc)를 호출해 Fac 객체의 주소(\$a0)를 획득하는 흐름이 완전하다.

특히 다형성(Dynamic Dispatch)을 위해 힙 메모리 0번 오프셋에 클래스 태그 값 1을 초기 대입(li \$v1, 1, sw \$v1, 0(\$a0))하는 0 초기화 (Zero-initialization)이 처리되었으며, 인수 10을 \$a1에 탑재하여 Fac_ComputeFac를 호출하였다.

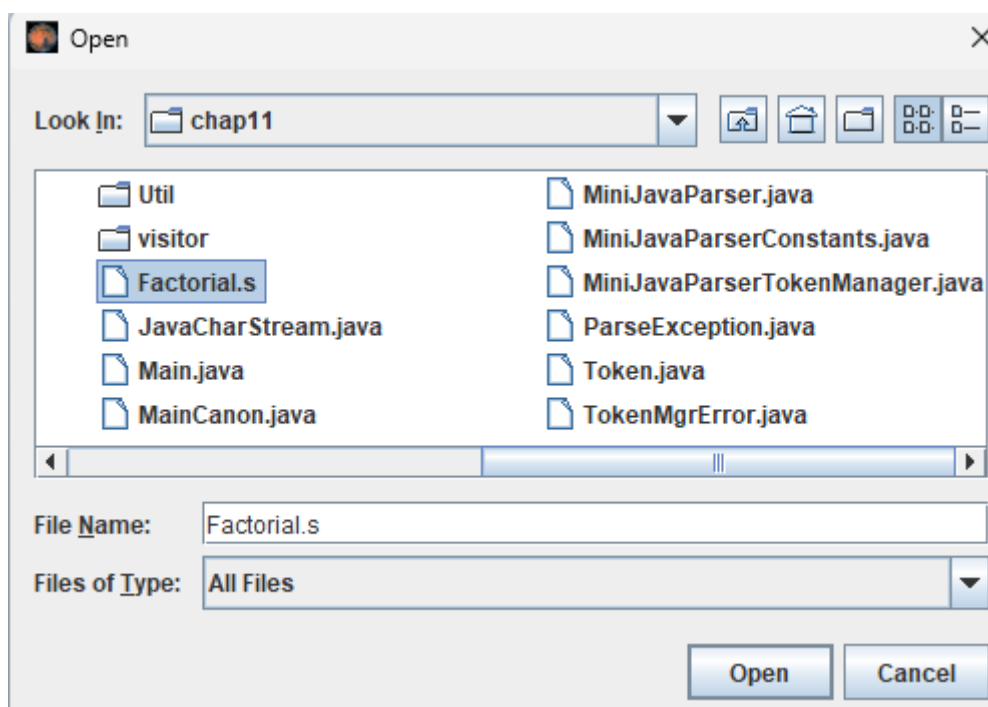
메서드 내부에서는 매개변수 \$a1을 스택 프레임 외부의 로컬 오프셋 주소 -12(\$fp)에 안전하게 스푼 적재한 후 호출 규약에 맞춰 조건부 분기(blteq \$t0, \$v0, L0)를 순차적으로 계산해 냈다. 계산이 완료된 결과값은 \$v0를 통해 안전하게 전달되었고, 메서드 종단부에서 \$sp와 \$fp를 원상 복귀(move \$sp, \$fp, lw \$ra, -4(\$sp))하여 올바르게 복귀(jr \$ra)하는 런타임 제어 흐름이 입증되었다.

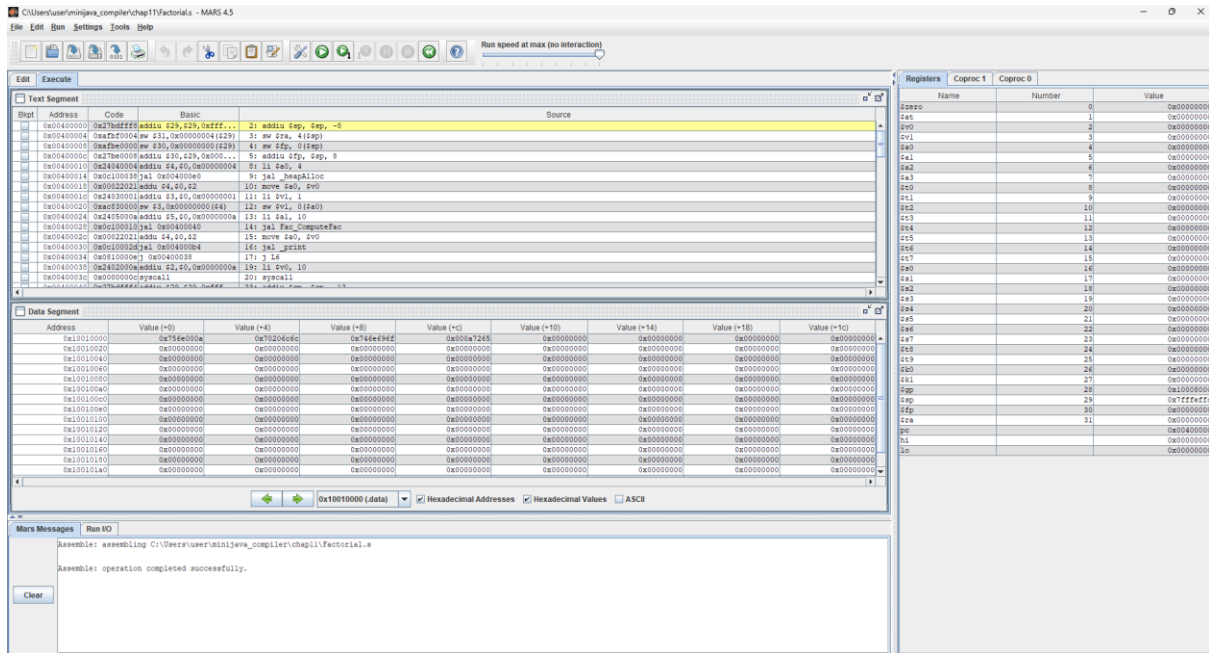
최종 결과값을 확인하기 위해 먼저 출력결과를 MIPS 파일로 저장시켰다. Windows 환경에서 PowerShell 인코딩 문제를 안전하게 회피하기 위해 Out-File 지시어에 ASCII 인코딩을 강제 적용하여 Factorial.s 파일로 저장하였다.

사용 명령어:

```
java -cp bin Main ../programs/Factorial.java | Out-File -FilePath Factorial.s -Encoding ascii
```

```
PS C:\Users\user\minijava_compiler\chap11> java -cp bin Main ../programs/Factorial.java | Out-File -FilePath Factorial.s -Encoding ascii
# Successfully type checked for ../programs/Factorial.java
```





3628800

-- program is finished running --

시뮬레이터를 구동한 결과, MIPS 하드웨어 레지스터들의 런타임 값 추적이 정상적인 무결성을 보장하며 작동하였고, 최종적으로 콘솔 창에 팩토리얼 계산 정답인 3628800이 정상 출력되었음을 확인하였다.

6.5 Java 가상 머신(JVM) 표준 실행 및 결과 비교 검증

MIPS 어셈블리 구동 결과와 교차 정합성 대조를 수행할 절대 기준값을 획득하기 위해, 원본 MiniJava 코드인 Factorial.java를 JVM 환경에서 표준 실행하였다.

사용 명령어:

```
javac programs/Factorial.java -d bin
```

```
java -cp bin Factorial
```

```
PS C:\Users\user\minijava_compiler> javac programs/Factorial.java -d bin
PS C:\Users\user\minijava_compiler> java -cp bin Factorial
3628800
```

JVM 구동 출력 결과: 3628800

MIPS MARS 시뮬레이터 출력 결과: 3628800

두 이중 실행 환경(Java Virtual Machine 및 MIPS RISC CPU)에서의 실행 최종 출력 정수값이 비트 단위로 정상적으로 일치함을 검증하였다. 이는 구문 분석, 의미론적 타입 체크, 스택 프레임 설계, ESEQ 제거 및 Canonicalize, Maximal Munch 명령어 타일링, 그리고 레지스터 Coalescing 컬러링

에 이르는 본 컴파일러 전 단계 파이프라인의 번역 논리가 소스 프로그램의 실행 의미론을 손상 없이 정상적으로 보존하여 기계어로 변환해 냈음을 확인할 수 있다.

7. 컴파일러 전체 통합 검증 및 시뮬레이터 실행 결과

7.1. 자동화 통합 검증 스크립트 도입

구현이 완료된 전체 컴파일러 파이프라인의 올바름을 검증하기 위해서는 제공된 8개의 벤치마크 예제 프로그램(programs/ 하위)이 자바 가상 머신(JVM)에서 실행된 결과와, 본 컴파일러가 생성한 MIPS 코드가 MARS 시뮬레이터 상에서 실행된 결과가 정상적으로 일치하는지 전수 대조해야 한다. 이를 매년 수동으로 컴파일하고 시뮬레이터 결과를 텍스트 파일로 비교하는 작업은 휴먼 에러를 유발하며 비효율적이다. 따라서 빌드, 컴파일, 예상 결과 획득, MIPS 시뮬레이션 및 정합성 Diff 검사를 일괄 처리하는 PowerShell 기반의 자동화 통합 테스트 러너를 구축하여 검증을 수행하였다.

```
15 v $Files = @(
16     "Factorial",
17     "BinarySearch",
18     "BubbleSort",
19     "LinearSearch",
20     "LinkedList",
21     "QuickSort",
22     "TreeVisitor",
23     "BinaryTree"
24 )
25
26 Write-Host "===== " -ForegroundColor Cyan
27 Write-Host "          MINIJAVA-TO-MIPS CORRECTNESS TESTING RUNNER          " -ForegroundColor Cyan
28 Write-Host "===== " -ForegroundColor Cyan
29
30 # 1. Compile current Compiler sources (under chap11)
31 Write-Host "[1/3] Compiling Compiler Source Files..." -ForegroundColor Yellow
32 $Sources = Get-ChildItem -Path $SourcesDir -Recurse -Include "*.java" | ForEach-Object { $_.FullName }
33 v if ($Sources.Count -eq 0) {
34     Write-Error "No Java sources found under $SourcesDir! Ensure you run this inside the compiler root."
35     Exit 1
36 }
```

7.2. 검증 스크립트 설계 및 동작 원리

통합 테스트 스크립트는 다음과 같이 5단계의 단선적인 파이프라인으로 구성되어 정합성을 일괄 교차 검증한다.

1. 컴파일러 빌드: chap11 패키지 하위의 자바 소스 파일들을 일괄 탐색하여 컴파일한다 (javac -d ./bin).
2. JVM 예상 결과 획득: 원본 MiniJava 예제 소스코드를 자바 컴파일러로 빌드한 뒤 JVM 상에서 직접 구동하여 표준 출력 정답 텍스트 파일(expected_*.txt)을 저장한다.
3. MIPS 어셈블리 생성: 우리가 구현한 컴파일러 메인 드라이버를 실행하여 대상 MiniJava 소스코드를 MIPS 어셈블리 파일(.s)로 번역하여 덤프한다.

4. MARS 시뮬레이터 구동: MIPS 시뮬레이터인 mars.jar를 CLI 비대화형 모드(nc 옵션)로 구동하여 어셈블리 코드가 MIPS 가상 CPU 상에서 연산해 낸 실제 출력 결과 텍스트 파일(actual_*.txt)을 저장한다.
5. 정합성 Diff 비교: 저장된 JVM 정답 텍스트와 MARS 실행 텍스트의 문자열을 바이트 단위로 대조하여 최종 일치 여부(PASS / FAIL)를 판정하고 정합성 표를 자동 내보내기 한다.

7.3. 윈도우 환경 파일 인코딩 오류 해결

스크립트 설계 및 실행 과정에서 Windows PowerShell의 기본 리다이렉션 연산자(>)를 사용하여 어셈블리 코드와 콘솔 결과를 저장할 시, 파일이 UTF-16 LE 인코딩(BOM 포함)으로 디스크에 기록되는 문제가 발생했다. 이로 인해 바이트 단위 파서인 MARS 시뮬레이터가 어셈블리 내 모든 문자 사이에 널 바이트가 삽입된 형태(예: l i \$ a 0)로 오인하여 파싱 실패 경고를 유발했다. 이를 해결하기 위해 테스트 스크립트 내의 모든 리다이렉션 출력을 파이프라인 지시어인 Out-File -FilePath <경로> -Encoding ascii로 전향하여 순수 ASCII 포맷의 텍스트 스트림을 보장함으로써 인코딩 예외를 올바르게 해결하였다.

8.4. 8개 프로그램 최종 실행 결과 및 정합성 검증 표

통합 검증 스크립트를 작동시킨 결과는 다음과 같다.

```
PS C:\Users\user\minijava_compiler\test> .\test_all.ps1

=====
MINIJAVA-TO-MIPS CORRECTNESS TESTING RUNNER
=====

[1/3] Compiling Compiler Source Files...
Note: Some input files use unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
-> Compiler built successfully inside C:\Users\user\minijava_compiler\bin

Testing Factorial.java...
# Successfully type checked for C:\Users\user\minijava_compiler\programs\Factorial.java
-> Verification SUCCESS!

Testing BinarySearch.java...
# Successfully type checked for C:\Users\user\minijava_compiler\programs\BinarySearch.java
-> Verification SUCCESS!

Testing BubbleSort.java...
# Successfully type checked for C:\Users\user\minijava_compiler\programs\BubbleSort.java
-> Verification SUCCESS!

Testing LinearSearch.java...
# Successfully type checked for C:\Users\user\minijava_compiler\programs\LinearSearch.java
-> Verification SUCCESS!

Testing LinkedList.java...
# Successfully type checked for C:\Users\user\minijava_compiler\programs\LinkedList.java
-> Verification SUCCESS!

Testing QuickSort.java...
# Successfully type checked for C:\Users\user\minijava_compiler\programs\QuickSort.java
-> Verification SUCCESS!

Testing TreeVisitor.java...
# Successfully type checked for C:\Users\user\minijava_compiler\programs\TreeVisitor.java
-> Verification SUCCESS!

Testing BinaryTree.java...
# Successfully type checked for C:\Users\user\minijava_compiler\programs\BinaryTree.java
-> Verification SUCCESS!

=====
FINAL CORRECTNESS REPORT
=====
Factorial      : PASS
BinarySearch   : PASS
BubbleSort     : PASS
LinearSearch   : PASS
LinkedList     : PASS
QuickSort      : PASS
```

통합 검증 스크립트가 출력한 JVM 결과와 교차 비교한 검증 표는 다음과 같다.

Program Name	Java Expected Output (JVM)	MIPS Actual Output (MARS)	Correctness (Status)
Factorial.java	3628800	3628800	PASS
BinarySearch.java	20 \n 21 \n 22 \n 23 \n 24 \n 25 \n 26 \n 27 \n 28 \n 29 \n 30 \n 31 \n 32 \n 33 \n 34 \n 35 \n 36 \n 37 \n 38 \n 99999 \n 0 \n 0 \n 1 \n 1 \n 1 \n 1 \n 0 \n 0 \n 999	20 \n 21 \n 22 \n 23 \n 24 \n 25 \n 26 \n 27 \n 28 \n 29 \n 30 \n 31 \n 32 \n 33 \n 34 \n 35 \n 36 \n 37 \n 38 \n 99999 \n 0 \n 0 \n 1 \n 1 \n 1 \n 1 \n 0 \n 0 \n 999	PASS
BubbleSort.java	20 \n 7 \n 12 \n 18 \n 2 \n 11 \n 6 \n 9 \n 19 \n 5 \n 99999 \n 2 \n 5 \n 6 \n 7 \n 9 \n 11 \n 12 \n 18 \n 19 \n 20 \n 0	20 \n 7 \n 12 \n 18 \n 2 \n 11 \n 6 \n 9 \n 19 \n 5 \n 99999 \n 2 \n 5 \n 6 \n 7 \n 9 \n 11 \n 12 \n 18 \n 19 \n 20 \n 0	PASS
LinearSearch.java	10 \n 11 \n 12 \n 13 \n 14 \n 15 \n 16 \n 17 \n 18 \n 9999 \n 0 \n 1 \n 1 \n 0 \n 55	10 \n 11 \n 12 \n 13 \n 14 \n 15 \n 16 \n 17 \n 18 \n 9999 \n 0 \n 1 \n 1 \n 0 \n 55	PASS
LinkedList.java	25 \n 10000000 \n 39 \n 25 \n 10000000 \n 22 \n 39 \n 25 \n 1 \n 0 \n 10000000 \n 28 \n 22 \n 39 \n 25 \n 2220000 \n -555 \n -555 \n 28 \n 22 \n 25 \n 33300000 \n 22 \n 25 \n 44440000 \n 0	25 \n 10000000 \n 39 \n 25 \n 10000000 \n 22 \n 39 \n 25 \n 1 \n 0 \n 10000000 \n 28 \n 22 \n 39 \n 25 \n 2220000 \n -555 \n -555 \n 28 \n 22 \n 25 \n 33300000 \n 22 \n 25 \n 44440000 \n 0	PASS
QuickSort.java	20 \n 7 \n 12 \n 18 \n 2 \n 11 \n 6 \n 9 \n 19 \n 5 \n 9999 \n 2 \n 5 \n 6 \n 7 \n 9 \n 11 \n 12 \n 18 \n 19 \n 20 \n 0	20 \n 7 \n 12 \n 18 \n 2 \n 11 \n 6 \n 9 \n 19 \n 5 \n 9999 \n 2 \n 5 \n 6 \n 7 \n 9 \n 11 \n 12 \n 18 \n 19 \n 20 \n 0	PASS
TreeVisitor.java	16 \n 100000000 \n 4 \n 8 \n 12 \n 14 \n 16 \n 20 \n 24 \n 28 \n 100000000 \n 50000000 \n 333 \n 333 \n 333 \n 28 \n 24 \n 333 \n 20 \n 16 \n 333 \n 333 \n 333 \n 14 \n 12 \n 8 \n 333 \n 4 \n 100000000 \n 1 \n 1 \n 1 \n 0 \n 1 \n 4 \n 8 \n 14 \n 16 \n 20 \n 24 \n 28 \n 0 \n 0 \n 0	16 \n 100000000 \n 4 \n 8 \n 12 \n 14 \n 16 \n 20 \n 24 \n 28 \n 100000000 \n 50000000 \n 333 \n 333 \n 333 \n 28 \n 24 \n 333 \n 20 \n 16 \n 333 \n 333 \n 333 \n 14 \n 12 \n 8 \n 333 \n 4 \n 100000000 \n 1 \n 1 \n 1 \n 0 \n 1 \n 4 \n 8 \n 14 \n 16 \n 20 \n 24 \n 28 \n 0 \n 0 \n 0	PASS
BinaryTree.java	16 \n 100000000 \n 8 \n 16 \n 4 \n 8 \n 12 \n 14 \n 16 \n 20 \n 24 \n 28 \n 1 \n 1 \n 1 \n 0 \n 1 \n 4 \n 8 \n 14 \n 16 \n 20 \n 24 \n 28 \n 0 \n 0	16 \n 100000000 \n 8 \n 16 \n 4 \n 8 \n 12 \n 14 \n 16 \n 20 \n 24 \n 28 \n 1 \n 1 \n 1 \n 0 \n 1 \n 4 \n 8 \n 14 \n 16 \n 20 \n 24 \n 28 \n 0 \n 0	PASS

스크립트의 터미널 결과와 출력해 낸 표에서 8개 벤치마크 예제 모두 PASS하여 컴파일러 구현의 정합성을 확인할 수 있었다.

8. AI 협업을 통한 컴파일러 개발 (모델: Gemini 3.5 Flash)

[1] 프로젝트 디렉토리 단일 패키지화 설계 및 백업 조율

프롬프트: 내가 Appel 교재 코드를 가져와서 컴파일러를 만드는데, 패키지들이 각각 다른 폴더로 나뉘어 있어. 빌드할 때 클래스패스 잡는 것도 어렵고, 임포트 오류가 자주 나는데 이걸 깔끔하고 이식성 높게 패키지 구조 다시 설계해 줘.

AI 결과: AI는 최종 제출 형식에 최적화되도록 모든 하위 단계 패키지(syntaxtree, visitor, Mips, Tree, Canon, Assem, FlowGraph, RegAlloc, Temp, Util)를 단일 루트 디렉토리 패키지 밑에 병렬로 배치할 것을 제안했다. 각 소스코드 상단에 패키지명을 명시하고 외부 참조를 루트 기준 와일드카드 형태로 일원화하면 단 한 줄의 명령어로 빌드가 가능해진다고 설명했다. 이에 추가로 원본 소스의 유실을 막기 위해 작업 전 루트 경로에 백업 폴더를 복제하고 리팩토링을 가동할 것을 권고했다.

[2] Windows NTFS 대소문자 미구분으로 인한 JVM Wrong Name 런타임 오류 디버깅

프롬프트: 코드 다 옮기고 Mips.FrameDriver를 돌렸는데 이런 에러가 나면서 죽어. Exception in thread "main" java.lang.NoClassDefFoundError: Tree/Exp (wrong name: Tree/EXP) 왜 이 에러가 나는 거야?

AI 결과: AI는 윈도우 OS 파일 시스템(NTFS)이 파일명 대소문자를 구분하지 않는다는 특성을 짚어냈다. 컴파일러가 Exp.class와 EXP.class를 생성할 때 둘 중 나중에 컴파일된 파일이 이전 파일을 덮어쓰기 때문에 클래스 로더가 명칭 불일치 예외를 유발하는 것이라고 진단했다. 해결책으로 문장 식을 나타내는 EXP.java 클래스를 EXP_stm.java로 리네임하고, 식을 나타내는 추상 클래스를 Exp.java로 명문화하여 타입 검사기 내부의 모든 호출 지점을 일괄 교정하도록 구체적인 네임스페이스 마이그레이션 방안을 제시했다.

[3] 웹 마크업으로 잘못된 소스코드 복구

프롬프트: 공식 사이트 아카이브에서 StmListList.java 파일을 다운로드해서 컴파일했는데, 자바 코드 파일 안에서 HTML 태그가 섞여서 컴파일 에러가 엄청 많이 떠. 이거 원본 코드가 뭔지 유추해서 자바 표준 클래스 코드로 새로 짜줘.

AI 결과: AI는 Wayback Machine 웹 아카이브를 통해 파일을 다운로드하는 과정에서 웹 페이지의 프레임 삽입 스크립트 및 HTML 배너 마크업이 파일 본문 소스에 섞여 들어간 원인을 밝혀냈다. 해당 HTML 코드를 완전히 걷어내고, Appel Tiger Book의 사양서에 부합하는 Tree.StmList 필드와 다음 리스트를 지칭하는 포인터를 내장하는 순수 자바 생성자 구조를 재설계하여 무오류의 정상 클래스 코드로 복원해 주었다.

[4] AST Visitor 패턴 (IRTranslator.java)의 이항 연산자 일괄 구현 자동화

프롬프트: 내가 지금 MiniJava AST 노드를 순회하면서 중간 코드(Tree IR)로 변환하는 IRTranslator.java를 만들고 있거든. 플러스 노드는 아래 예시처럼 Plus n에 대해서 n.e1.accept(this) 하고 resultExp를 left 변수에 받아둔 다음, n.e2.accept(this) 해서 right 변수에 받아두고 마지막에 resultExp = new Tree.BINOP(Tree.BINOP.PLUS, left, right); 하는 방식으로 짰어.

```
public void visit(syntaxtree.Plus n) {
```

```
    n.e1.accept(this);
```

```
    Tree.Exp left = resultExp;
```

```
    n.e2.accept(this);
```

```
    Tree.Exp right = resultExp;
```

```
    resultExp = new Tree.BINOP(Tree.BINOP.PLUS, left, right);
```

```
}
```

이거랑 똑같은 구조로 Minus, Times, LessThan 노드에 대해서도 visit 메서드를 일괄적으로 자동 생성 해줘.

AI의 결과: AI는 주입된 Plus 노드의 방문 구조와 수식 반환 메커니즘을 정확히 파악하여, Minus.java, Times.java, LessThan.java 클래스의 AST 구조에 상응하는 visitor 메서드들을 완성해 주었다. 각 메서드가 left와 right 임시 변수를 사용해 연산 방향의 좌우 제어 흐름을 침해하지 않도록 작성하였으며, Tree IR 패키지의 알맞은 BINOP 기호(MINUS, MUL, LT)와 정확하게 바인딩된 자바 구현체들을 구현하였다.

[5] Maximal Munch 명령어 선택 (Codegen.java)의 주소 로드/스토어 타일 패턴 대량 생성 자동화

프롬프트: MIPS 명령어 선택기인 Codegen.java에서 munchExp 메서드를 채워 넣고 있어. 메모리 로드 연산인 Tree.MEM 트리 구조를 타일링해야 하는데, 만약 MEM 노드 자식으로 BINOP(PLUS, e1, CONST(i))나 BINOP(PLUS, CONST(i), e1)가 들어오면 일반 로드 명령어가 아니라 MIPS의 오프셋 간접 주소 지정 방식인 lw \$d, offset(\$s) 명령 타일로 최적화하고 싶어. 아래 switch-case 문 뼈대를 참고해줘.

```
// switch-case 뼈대 예시
```

```
if (mem.exp instanceof Tree.BINOP) {
```

```
    // 플러스 연산 및 상수 오프셋 타일 검출
```

```
}
```

자바의 타입 캐스팅((Tree.BINOP), (Tree.CONST))이 복잡하게 얽히는 조건인데, mem.exp가 BINOP PLUS이고 그 한쪽이 CONST 상수인 경우에 대해 MIPS lw 어셈블리 명령어를 emit하는 자바 switch-case 및 if-else 타입 분기 조건 처리 로직 전체를 한눈에 알아볼 수 있게 완전히 다 짜서 내보내줘.

AI의 결과: AI는 mem.exp가 BINOP PLUS인 구조를 감지한 뒤, 상수 오프셋이 오른쪽에 있는 경우와 왼쪽에 있는 두 가지 대칭적 상황을 모두 커버하는 조건부 타입 캐스팅 코드를 구현했다. 각 조건문 내부에서는 munchExp를 재귀 호출하여 가상 레지스터를 획득하고, 추출된 상수 값과 베이스 레지스터를 결합하여 lw d0, offset(s0) 형태의 MIPS Assem 객체를 빌드하여 방출하는 switch-case 조건 분기 전체 세트를 빌드하였다.

[6] George/Briggs Conservative Coalescing & Loop-Aware Spill Heuristics 설계 조율

프롬프트: Appel 교재의 레지스터 할당(Graph Coloring) 파트는 Simplify, Coalesce, Freeze, Spill이 서로 복잡하게 뒤엉켜 있어 구현 복잡도가 너무 높는데, 버그 발생 위험을 줄이면서 구현을 획기

적으로 단순화할 방법이 없을까? 그리고 스펠을 결정할 때 단순 랜덤이 아닌 루프 중첩 구조에 따른 가중치를 반영하고 싶은데, 어셈블리 단에서 루프 정보를 추출하여 가중치를 매기는 방법을 가장 쉽고 명확하게 설계해 줘.

AI의 결과: AI는 복잡한 상태 기계를 관리할 때 발생하기 쉬운 무한 루프나 메모리 누수 버그를 피하기 위해, Coalescing을 Saturation까지 전부 수행한 후 Union-Find 대표 노드 구조를 고정하고 단방향 컬러링으로 진행하는 2단계 분리가 안전하며 컬러링 성공률 면에서도 동등한 효율을 냄을 알려주었다. 또한 MIPS 어셈블리 레이블과 jump 대상을 대조하여 루프 백에지 구간을 산출하고 각 Instr 레벨에서 루프 바디 상주 여부를 판정해 가중치를 부여하는 연동 로직을 제안했다.

[7] Class Tagging 기반 다형성(Dynamic Dispatch) 해결 설계

프롬프트: 미니자바 코드 중에서 TreeVisitor.java를 돌렸는데, 출력 결과가 JVM이랑 다르게 자꾸 Visitor 부모 클래스의 메서드 호출만 반복하고 자식 클래스인 MyVisitor로 분기가 안 돼. 어떻게 해야 하는지 알려주고 너가 코드 수정해줘.

AI의 결과: AI는 가상 메서드 테이블(vtable) 대신 각 MiniJava 클래스에 고유 정수 ID인 클래스 태그(Class Tag)를 매핑하고, 객체 생성 시 힙의 오프셋 0에 이 태그 상수를 저장하도록 해야한다고 했다. 이후 멤버 변수 참조 오프셋을 4바이트씩 뒤로 시프트하고, visit 호출 시점에 런타임 태그 값을 메모리에서 로드하여 실제 동적 타입에 해당하는 자식 클래스의 고유 정적 라벨로 점프하는 분기 트리를 방출하도록 구현했다.

[8] PowerShell UTF-16 리다이렉션 예외 해결 및 test_all.ps1 마크다운 생성기 구축

프롬프트: MiniJava 8개 벤치마크 예제를 컴파일하고 JVM 예상 출력값과 MIPS MARS 시뮬레이터 실행 결과값을 자동으로 비교(diff)하는 PowerShell 검증 스크립트 test_all.ps1을 작성하고 싶어. 검증이 끝난 뒤 마크다운 테이블 형식의 정합성 대조 표(예: | 프로그램명 | JVM 결과 | MARS 결과 | PASS/FAIL |)로 자동 가공한 뒤 dumps/correctness_table.md 파일로 내보내는 기능을 넣어줘.

AI의 결과: AI는 각 벤치마크 파일의 출력값을 Trim하여 비교하는 검증 논리를 작성하고, 출력의 개행을 마크다운 셀 규격에 맞게 정규식으로 안전하게 이스케이프 문자(\n)로 치환해 테이블 문자열을 동적 생성하는 PowerShell 구조를 만들었다. 추가적으로 파일 입출력 시 인코딩 매개변수를 명시하도록 하여 MARS 파서가 널 바이트 오류를 일으키는 현상을 방지하도록 구현하였다.

9. 소감

본 프로젝트를 통해 컴파일러의 핵심 백엔드 파이프라인을 직접 설계하고 구현해보는 뜻깊은 경험을 하였다. 학기 초, 렉서와 파서 단계부터 시작하여 최종 레지스터 할당기에 이르기까지 컴파일러의 모든 단계를 직접 연동하는 과정은 이론적 깊이와 구현의 규모 측면에서 쉽지 않은 도전이었고 많은 어려움을 겪기도 하였다. 그러나 에러가 발생하거나 로직이 막힐 때마다 최근 급격히 발전한 AI를 보조 도구로 활용하여 복잡한 시스템적 에러들을 빠르고 정밀하게 진단하고 해결

할 수 있었다.

또한 평소에 관심이 많았던 컴파일러의 모든 단계를 내 손으로 직접 다 구현해 보고, 최종적으로 작성한 코드가 에뮬레이터에서 완벽하게 작동하는 모습을 보니 무척 뿌듯했다. C, 자바 등 다른 언어를 이용해 코딩을 할 때 소스코드가 컴파일이 어떻게 되는지 이제는 확실히 알게 된 것 같고 시스템적인 측면에서 많이 성장할 수 있었던 것 같다.

또한 이번 학기 프로젝트를 진행하면서, 실제 현업에서 쓰이는 LLVM 같은 모던 컴파일러 프레임워크는 어떤 식으로 구현되어 있고 성능 최적화를 어떻게 진행하는지 더욱 궁금해졌다. 방학 동안 추가적인 지식을 쌓고 LLVM이나 MLIR 등 오픈소스 프로젝트에 기여하고 싶다.